`Spam Detection Using Fine-Tuned BERT`

`Dataset:` The SMS Spam Collection is a set of SMS tagged messages that have been collected for SMS Spam research. It contains one set of SMS messages in English of 5,574 messages, tagged acording being ham (legitimate) or spam.

`Objective:` Build an AI model that can classify messages as spam or legitimate using BERT model

Table of Contents

1. Loading Data
2. Data Preparation
3. Exploratory Data Analysis (EDA)
4. Original Data
   A. Train/Test Splitting
   B. Model Building
   C. Model Evaluation
5. Hyperparameter Tuning with Keras Tuner (Original Data)
   A. Model Building
   B. Model Evaluation
   C. Additional Metrics (ROC-AUC) for BERT
6. Downsampling Data
   A. Train/Test Splitting
   B. Model Building
   C. Model Evaluation
7. Hyperparameter Tuning with Keras Tuner (Downsampled Data)
   A. Model Building
   B. Model Evaluation
   C. Additional Metrics (ROC-AUC) for BERT
8. Upsampling Data
   A. Train/Test Splitting
   B. Model Building
   C. Model Evaluation
9. Hyperparameter Tuning with Keras Tuner (Upsampled Data)
   A. Model Building
   B. Model Evaluation
   C. Additional Metrics (ROC-AUC) for BERT
10. Class Weights
   A. Model Building
   B. Model Evaluation
11. Hyperparameter Tuning with Keras Tuner (Class Weights)
   A. Model Building
   B. Model Evaluation

C. Additional Metrics (ROC-AUC) for BERT
12. Custom Loss
A. Model Building
B. Model Evaluation
13. Hyperparameter Tuning with Keras Tuner (Custom Loss)
A. Model Building
B. Model Evaluation
C. Additional Metrics (ROC-AUC) for BERT
14. Performance Comparison of Fine-Tuned BERT
15. Performance Comparison of Hyperparameter-Tuned BERT
16. Inference Comparison of Fine-Tuned BERT
17. Inference Comparison of Hyperparameter-Tuned BERT
18. Conclusion
19. Deployment
20. References

Loading Data

Table of Contents

In [1]:
```
# import modules
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
import tensorflow_hub as hub
import tensorflow_text as text
import os
import joblib
import warnings #ignore warning
warnings.filterwarnings("ignore")
```

In [2]:
```
# Define file path
filepath ="D://Portfolio//Internship//AFAME//Project details//Spam SMS Detection//spam.csv"
# Read the CSV file into a DataFrame with specified encoding
data = pd.read_csv(filepath, encoding='latin1')
# Display the first few rows of the DataFrame
data.head()
```

Out[2]:

	v1	v2	Unnamed: 2	Unnamed: 3	Unnamed: 4
0	ham	Go until jurong point, crazy.. Available only ...	NaN	NaN	NaN
1	ham	Ok lar... Joking wif u oni...	NaN	NaN	NaN
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	NaN	NaN	NaN
3	ham	U dun say so early hor... U c already then say...	NaN	NaN	NaN
4	ham	Nah I don't think he goes to usf, he lives aro...	NaN	NaN	NaN

In [3]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   v1          5572 non-null   object
 1   v2          5572 non-null   object
 2   Unnamed: 2  50 non-null     object
 3   Unnamed: 3  12 non-null     object
 4   Unnamed: 4  6 non-null      object
dtypes: object(5)
memory usage: 217.8+ KB
```

Data Preparation

Table of Contents

In [4]:
```
# removing excess unnecessary column
data = data.loc[:, ~data.columns.str.contains('^Unnamed')]
data.head()
```

Out[4]:

	v1	v2
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

In [5]:
```
data.rename(columns={'v1': 'Class', 'v2': 'Text'}, inplace=True)
```

In [6]:
```
data['Class'] = data['Class'].map({'ham':0, 'spam':1})
data.head()
```

Out[6]:

	Class	Text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

In [7]:
```
# Checking null values
data.isnull().sum()
```

Out[7]:
```
Class    0
Text     0
dtype: int64
```

In [8]:
```
data.duplicated().sum()
```

Out[8]:
```
403
```

We are retaining the duplicate values as they are crucial for our task of identifying spam SMS messages.

In [9]:
```
# Viewing values in 'v1' column
data['Class'].value_counts()
```

Out[9]:
```
Class
0    4825
1     747
Name: count, dtype: int64
```

In [10]:
```
data.groupby('Class').describe()
```

Out[10]:

				Text
	count	unique	top	freq
Class				
0	4825	4516	Sorry, I'll call later	30
1	747	653	Please call our customer service representativ...	4

In [11]:
```
# Viewing the imbalanced rate
747/4825
```

Out[11]:
```
0.15481865284974095
```

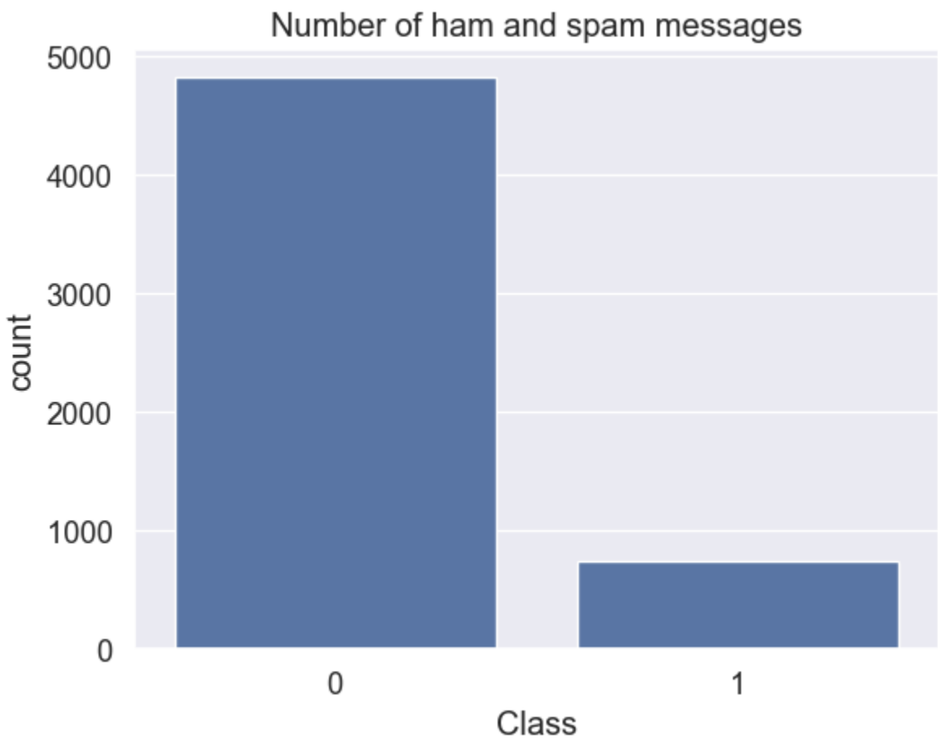
In [12]:
```
# Viewing unique values in 'v2'
data['Text'].nunique()
```

Out[12]:
```
5169
```

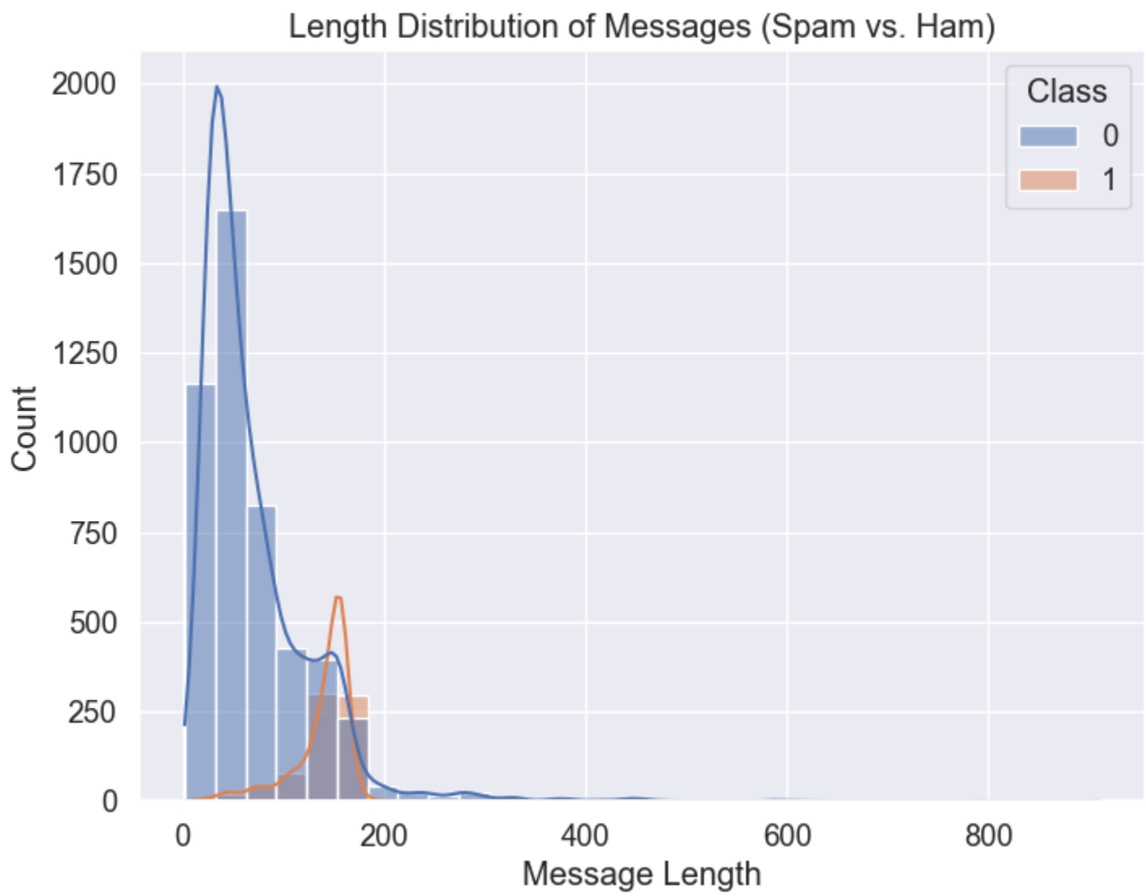
Exploratory Data Analysis (EDA)

Table of Contents

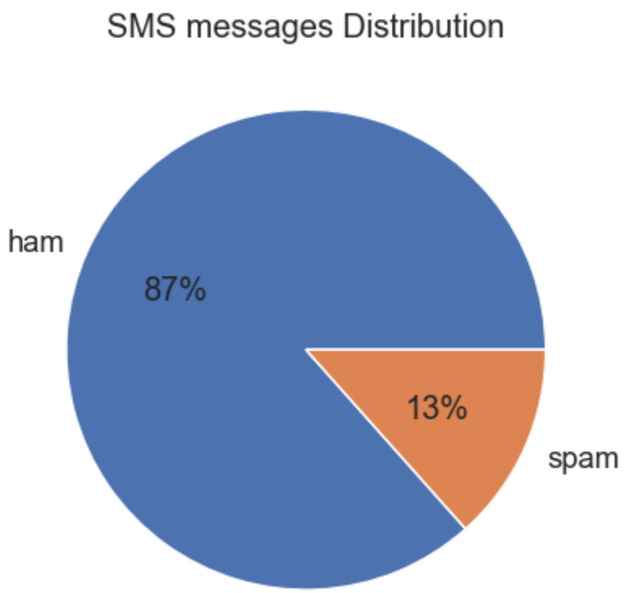
In [13]:
```
sns.set(style = "darkgrid" , font_scale = 1.2)
sns.countplot(data=data, x= 'Class').set_title("Number of ham and spam messages")
plt.show()
```



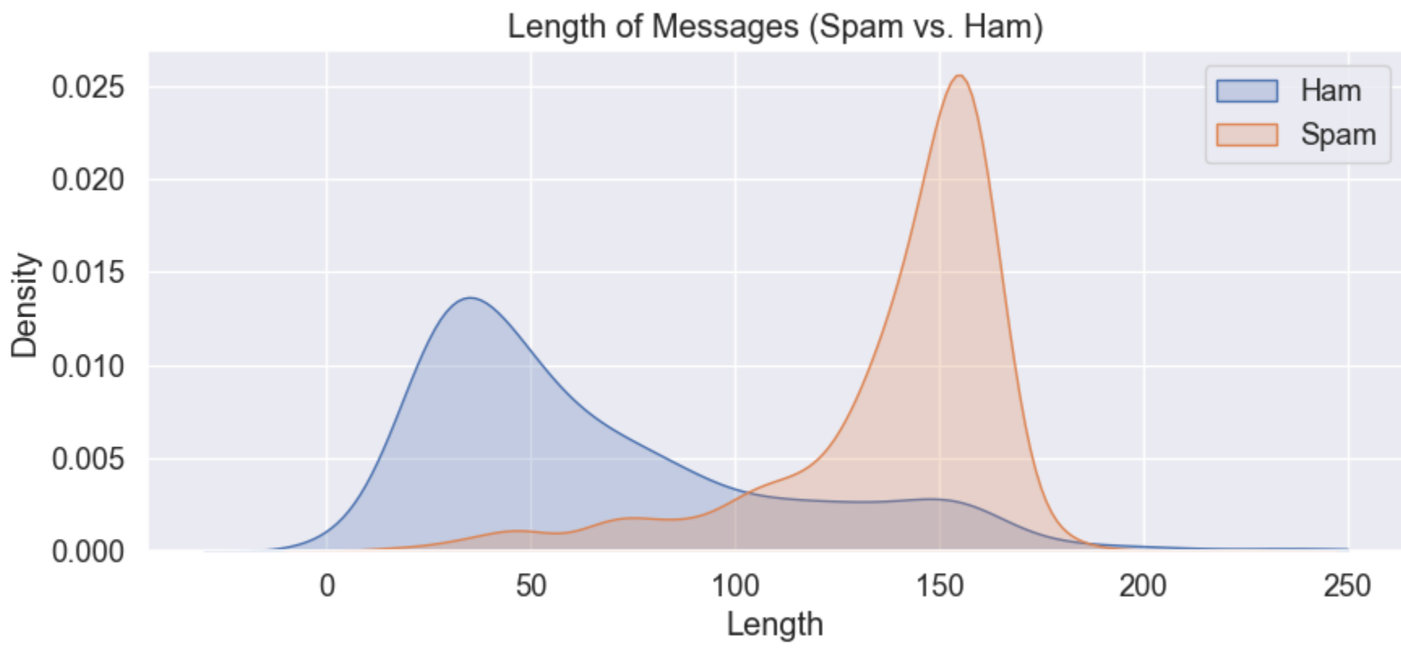
In [14]:
```
# Plot the distribution of message lengths
plt.figure(figsize=(8, 6))
sns.histplot(x=data['Text'].str.len(), bins=30, hue=data['Class'], kde=True)
plt.title('Length Distribution of Messages (Spam vs. Ham)')
plt.xlabel('Message Length')
plt.ylabel('Count')
plt.show()
```



In [15]:
```
sms = pd.value_counts(data["Class"], sort=True)
sms.plot(kind="pie", labels=["ham", "spam"], autopct="%1.0f%%")

plt.title("SMS messages Distribution")
plt.ylabel("")
plt.show()
```



In [16]:
```
# Calculate lengths of messages directly in the plotting function
_, ax = plt.subplots(figsize=(10, 4))
sns.kdeplot(data.loc[data.Class == 0, 'Text'].str.len(), shade=True, label='Ham', clip=(-50, 250))
sns.kdeplot(data.loc[data.Class == 1, 'Text'].str.len(), shade=True, label='Spam', clip=(-50, 250))

# Set axis labels and title
ax.set(
    xlabel='Length',
    ylabel='Density',
    title='Length of Messages (Spam vs. Ham)'
)
ax.legend(loc='upper right')

# Show plot
plt.show()
```



Original Data

Table of Contents

Train/Test Spliting

Table of Contents

In [17]:
```
X = data['Text']
y = data['Class']
```

In [18]:
```
X.shape
```

Out[18]:
```
(5572,)
```

In [19]:
```
y.shape
```

Out[19]:
```
(5572,)
```

In [20]:
```
print(f'Length of X: {len(X)}')
print(f'Length of y: {len(y)}')
```

```
Length of X: 5572
Length of y: 5572
```

In [21]:
```
from sklearn.model_selection import train_test_split
random_seed = 42

# Split data into training and temporary sets (60% train, 40% temp)
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=random_seed, stratify = y)

# Split temporary set into validation and test sets (20% val, 20% test)
```

```
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=random_seed, stratify = y_temp)

print(f'Training data: {len(X_train)}, {len(y_train)}')
print(f'Validation data: {len(X_val)}, {len(y_val)}')
print(f'Testing data: {len(X_test)}, {len(y_test)}')
```

```
Training data: 3900, 3900
Validation data: 836, 836
Testing data: 836, 836
```

Model Building

Table of Contents

In [22]:
```
%%time
bert_preprocessor = hub.KerasLayer('https://tfhub.dev/tensorflow/bert_en_uncased_preprocess/3') # Loading Bert Preprocessor
bert_encoder = hub.KerasLayer('https://tfhub.dev/tensorflow/bert_en_uncased_L-12_H-768_A-12/4') # Loading Bert Encoder
```

```
CPU times: total: 9.7 s
Wall time: 14.3 s
```

In [23]:
```
%%time
from tensorflow.keras.layers import Input, Dropout, Dense, BatchNormalization
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam  # Import the optimizer

# Functional BERT layers
text_input = Input(shape=(), dtype=tf.string, name='Inputs')
encoder_inputs = bert_preprocessor(text_input)
embbed = bert_encoder(encoder_inputs)

# Neural Network layers
dropout1 = Dropout(0.1, name='Dropout1')(embbed['pooled_output'])
dense1 = Dense(128, activation='relu', name='Dense1')(dropout1)
batch_norm1 = BatchNormalization(name='BatchNorm1')(dense1)
dropout2 = Dropout(0.1, name='Dropout2')(batch_norm1)
dense2 = Dense(64, activation='relu', name='Dense2')(dropout2)
batch_norm2 = BatchNormalization(name='BatchNorm2')(dense2)
dropout3 = Dropout(0.1, name='Dropout3')(batch_norm2)
outputs = Dense(1, activation='sigmoid', name='Output')(dropout3)

# Creating final model
model = Model(inputs=[text_input], outputs=[outputs])
```

```
CPU times: total: 438 ms
Wall time: 630 ms
```

In [24]:
```
model.summary()
```

```
Model: "model"
__________________________________________________________________________________________________
 Layer (type)                   Output Shape         Param #     Connected to
==================================================================================================
 Inputs (InputLayer)            [(None,)]            0           []

 keras_layer (KerasLayer)       {'input_word_ids':   0           ['Inputs[0][0]']
                                (None, 128),
                                 'input_type_ids':
                                (None, 128),
                                 'input_mask': (Non
                                e, 128)}

 keras_layer_1 (KerasLayer)     {'encoder_outputs':  109482241   ['keras_layer[0][0]',
                                 [(None, 128, 768),               'keras_layer[0][1]',
                                 (None, 128, 768),                'keras_layer[0][2]']
                                 (None, 128, 768),
                                 (None, 128, 768),
                                 (None, 128, 768),
                                 (None, 128, 768),
                                 (None, 128, 768),
                                 (None, 128, 768),
                                 (None, 128, 768),
                                 (None, 128, 768),
                                 (None, 128, 768),
                                 (None, 128, 768),
                                 (None, 128, 768)],
                                 'sequence_output':
                                 (None, 128, 768),
                                 'pooled_output': (
                                None, 768),
                                 'default': (None,
                                768)}

 Dropout1 (Dropout)             (None, 768)          0           ['keras_layer_1[0][13]']

 Dense1 (Dense)                 (None, 128)          98432       ['Dropout1[0][0]']

 BatchNorm1 (BatchNormalization  (None, 128)         512         ['Dense1[0][0]']
 )

 Dropout2 (Dropout)             (None, 128)          0           ['BatchNorm1[0][0]']

 Dense2 (Dense)                 (None, 64)           8256        ['Dropout2[0][0]']

 BatchNorm2 (BatchNormalization  (None, 64)          256         ['Dense2[0][0]']
 )

 Dropout3 (Dropout)             (None, 64)           0           ['BatchNorm2[0][0]']

 Output (Dense)                 (None, 1)            65          ['Dropout3[0][0]']

==================================================================================================
Total params: 109,589,762
Trainable params: 107,137
Non-trainable params: 109,482,625
__________________________________________________________________________________________________
```

In [25]:

```
metrics = [
    tf.keras.metrics.BinaryAccuracy(name='accuracy'),
    tf.keras.metrics.Precision(name='precision'),
```

```
    tf.keras.metrics.Recall(name='recall')
]

model.compile(optimizer=Adam(learning_rate=0.001),
              loss='binary_crossentropy',
              metrics = metrics)
```

In [26]:
```
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
# Define callbacks
early_stopping = EarlyStopping(monitor='val_loss',
                               patience=3,
                               restore_best_weights=True)

# Learning rate scheduler
lr_scheduler = ReduceLROnPlateau(monitor='val_loss',
                                 factor=0.2,
                                 patience=2,
                                 min_lr=1e-7)
```

In [27]:
```
%%time
# Fit the model
history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_val, y_val),
                    callbacks=[early_stopping, lr_scheduler])
```

```
Epoch 1/20
122/122 [==============================] - 74s 517ms/step - loss: 0.4355 - accuracy: 0.8223 - precision: 0.4224 - recall: 0.8853 - val_loss: 0.3146 - val_accuracy: 0.9390 - val_precision: 1.00
00 - val_recall: 0.5446 - lr: 0.0010
Epoch 2/20
122/122 [==============================] - 61s 504ms/step - loss: 0.1806 - accuracy: 0.9515 - precision: 0.7840 - recall: 0.8815 - val_loss: 0.2786 - val_accuracy: 0.9139 - val_precision: 0.61
11 - val_recall: 0.9821 - lr: 0.0010
Epoch 3/20
122/122 [==============================] - 61s 504ms/step - loss: 0.1252 - accuracy: 0.9636 - precision: 0.8772 - recall: 0.8470 - val_loss: 0.0935 - val_accuracy: 0.9665 - val_precision: 1.00
00 - val_recall: 0.7500 - lr: 0.0010
Epoch 4/20
122/122 [==============================] - 62s 509ms/step - loss: 0.1161 - accuracy: 0.9618 - precision: 0.8681 - recall: 0.8432 - val_loss: 0.0880 - val_accuracy: 0.9701 - val_precision: 0.86
55 - val_recall: 0.9196 - lr: 0.0010
Epoch 5/20
122/122 [==============================] - 63s 515ms/step - loss: 0.1016 - accuracy: 0.9679 - precision: 0.8964 - recall: 0.8604 - val_loss: 0.0739 - val_accuracy: 0.9809 - val_precision: 0.96
15 - val_recall: 0.8929 - lr: 0.0010
Epoch 6/20
122/122 [==============================] - 63s 519ms/step - loss: 0.0994 - accuracy: 0.9654 - precision: 0.8896 - recall: 0.8470 - val_loss: 0.0542 - val_accuracy: 0.9844 - val_precision: 0.94
59 - val_recall: 0.9375 - lr: 0.0010
Epoch 7/20
122/122 [==============================] - 63s 514ms/step - loss: 0.0953 - accuracy: 0.9674 - precision: 0.8944 - recall: 0.8585 - val_loss: 0.0522 - val_accuracy: 0.9809 - val_precision: 0.94
44 - val_recall: 0.9107 - lr: 0.0010
Epoch 8/20
122/122 [==============================] - 62s 511ms/step - loss: 0.0780 - accuracy: 0.9738 - precision: 0.9168 - recall: 0.8853 - val_loss: 0.1218 - val_accuracy: 0.9533 - val_precision: 0.75
17 - val_recall: 0.9732 - lr: 0.0010
Epoch 9/20
122/122 [==============================] - 62s 512ms/step - loss: 0.0849 - accuracy: 0.9715 - precision: 0.9221 - recall: 0.8604 - val_loss: 0.0782 - val_accuracy: 0.9809 - val_precision: 0.92
11 - val_recall: 0.9375 - lr: 0.0010
Epoch 10/20
122/122 [==============================] - 63s 519ms/step - loss: 0.0803 - accuracy: 0.9733 - precision: 0.9249 - recall: 0.8719 - val_loss: 0.0621 - val_accuracy: 0.9797 - val_precision: 1.00
00 - val_recall: 0.8482 - lr: 2.0000e-04
CPU times: total: 7min 37s
Wall time: 10min 35s
```

Model Evaluation

Table of Contents

In [28]:
```
# Evaluate the model
results = model.evaluate(X_test, y_test)
print(f"Test Loss: {results[0]}")
print(f"Test Accuracy: {results[1]}")
```

```
27/27 [==============================] - 11s 407ms/step - loss: 0.0446 - accuracy: 0.9892 - precision: 0.9725 - recall: 0.9464
Test Loss: 0.044614627957344055
Test Accuracy: 0.989234447479248
```

In [29]:
```
y_predict = model.predict(X_test)
y_predict = y_predict.flatten()
```

```
27/27 [==============================] - 12s 411ms/step
```

In [30]:
```
y_predict = np.where(y_predict > 0.5,1,0)
y_predict
```

Out[30]:
```
array([0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0,
       0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1])
```

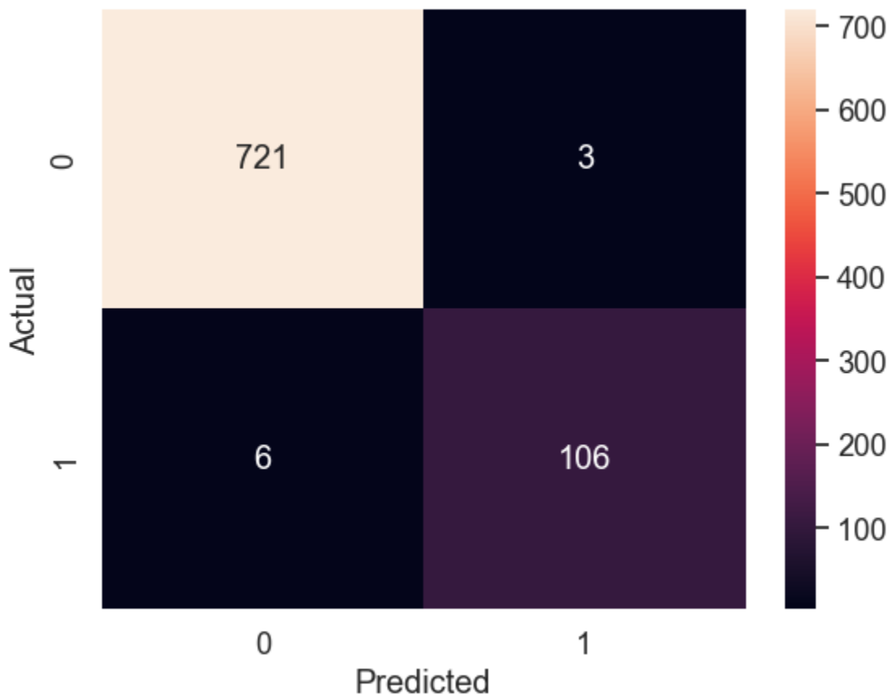
In [31]:
```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_test, y_predict)
cm
```

Out[31]:
```
array([[721,   3],
       [  6, 106]], dtype=int64)
```

In [32]:
```
sns.heatmap(cm, annot=True, fmt = 'd')
plt.xlabel('Predicted')
plt.ylabel('Actual')
```

Out[32]:
```
Text(43.25, 0.5, 'Actual')
```



In [33]:
```
from sklearn.metrics import classification_report

nml = classification_report(y_test, y_predict, target_names=['Ham', 'Spam'])
print(nml)
```

```
              precision    recall  f1-score   support

         Ham       0.99      1.00      0.99       724
        Spam       0.97      0.95      0.96       112

    accuracy                           0.99       836
   macro avg       0.98      0.97      0.98       836
weighted avg       0.99      0.99      0.99       836
```

Inference

In [34]:
```
# Actual real examples
reviews = [
    'You will be hired at managerial roles in top companies',
```

```
    'Citrusbug Technolabs is hiring for AI/ML Engineer + 14 new Fresher Data Science jobs',
    'Job opportunity from Wipro just for You!',
    'Dear Congratulations - Get Your Job Offer Letter @ Cognizant',
    'Practice Coding with A Very Big Sum'
]

# Detection of examples
nml_inf = model.predict(reviews)
nml_inf
```

```
1/1 [==============================] - 1s 756ms/step
```

Out[34]:

```
array([[0.01657198],
       [0.33283165],
       [0.06613444],
       [0.45205396],
       [0.00761542]], dtype=float32)
```

Hyperparameter Tuning with Keras Tuner (Original Data)

Table of Contents

In [35]:

```
%%time
import kerastuner as kt

# Function to compile your existing model with hyperparameters
def compile_model(hp):
    # Tune the learning rate
    learning_rate = hp.Choice('learning_rate', values=[1e-2, 1e-3, 1e-4])

    # Compile the model with the chosen learning rate
    model.compile(optimizer = Adam(learning_rate=learning_rate),
                  loss = 'binary_crossentropy',
                  metrics = metrics)
    return model

# Initialize Keras Tuner with the existing model
tuner = kt.Hyperband(
    compile_model,
    objective='val_accuracy',
    max_epochs=5,
    factor=3,
    directory=os.getcwd(),
    project_name='nml_model'
)

# Perform hyperparameter tuning
tuner.search(X_train, y_train, epochs=5, validation_data=(X_val, y_val))

# Get the best model
best_model_nml = tuner.get_best_models(num_models=1)[0]
```

```
Trial 3 Complete [00h 02m 12s]
val_accuracy: 0.9796651005744934

Best val_accuracy So Far: 0.980861246585846
Total elapsed time: 00h 06m 36s
CPU times: total: 4min 50s
Wall time: 6min 37s
```

In [36]:
```
# Evaluate the best model on the validation set
hp_results_val = best_model_nml.evaluate(X_val, y_val)
print(f"\nValidation Loss: {hp_results_val[0]}")
print(f"Validation Accuracy: {hp_results_val[1]}")
```

```
27/27 [==============================] - 12s 405ms/step - loss: 0.0516 - accuracy: 0.9797 - precision: 0.9279 - recall: 0.9196

Validation Loss: 0.05157732218503952
Validation Accuracy: 0.9796651005744934
```

In [37]:
```
# Evaluate the best model on the validation set
hp_results_val = best_model_nml.evaluate(X_test, y_test)
print(f"\Test Loss: {hp_results_val[0]}")
print(f"Test Accuracy: {hp_results_val[1]}")
```

```
27/27 [==============================] - 11s 406ms/step - loss: 0.0432 - accuracy: 0.9880 - precision: 0.9554 - recall: 0.9554
\Test Loss: 0.04315700754523277
Test Accuracy: 0.9880383014678955
```

Model Building

Table of Contents

In [38]:
```
%%time
# Fitting best model on X_train, y_train, X_val, y_val, X_test, y_test
best_model_nml.fit(X_train, y_train, epochs=20,
                   validation_data=(X_val, y_val),
                   callbacks=[early_stopping, lr_scheduler])
```

```
Epoch 1/20
122/122 [==============================] - 66s 514ms/step - loss: 0.0862 - accuracy: 0.9721 - precision: 0.9224 - recall: 0.8642 - val_loss: 0.0441 - val_accuracy: 0.9868 - val_precision: 0.96
33 - val_recall: 0.9375 - lr: 0.0010
Epoch 2/20
122/122 [==============================] - 62s 512ms/step - loss: 0.0817 - accuracy: 0.9723 - precision: 0.9125 - recall: 0.8776 - val_loss: 0.0595 - val_accuracy: 0.9773 - val_precision: 0.89
08 - val_recall: 0.9464 - lr: 0.0010
Epoch 3/20
122/122 [==============================] - 63s 520ms/step - loss: 0.0743 - accuracy: 0.9754 - precision: 0.9228 - recall: 0.8910 - val_loss: 0.0663 - val_accuracy: 0.9809 - val_precision: 0.89
34 - val_recall: 0.9732 - lr: 0.0010
Epoch 4/20
122/122 [==============================] - 62s 513ms/step - loss: 0.0701 - accuracy: 0.9749 - precision: 0.9208 - recall: 0.8891 - val_loss: 0.0452 - val_accuracy: 0.9844 - val_precision: 0.97
14 - val_recall: 0.9107 - lr: 2.0000e-04
CPU times: total: 3min 2s
Wall time: 4min 13s
```

Out[38]:
```
<keras.callbacks.History at 0x219ec009c90>
```

Model Evaluation

Table of Contents

In [39]:
```
y_predict_hp = best_model_nml.predict(X_test)
y_predict_hp = y_predict_hp.flatten()
```

```
27/27 [==============================] - 12s 412ms/step
```

In [40]:
```
y_predict_hp = np.where(y_predict_hp > 0.5,1,0)
y_predict_hp
```

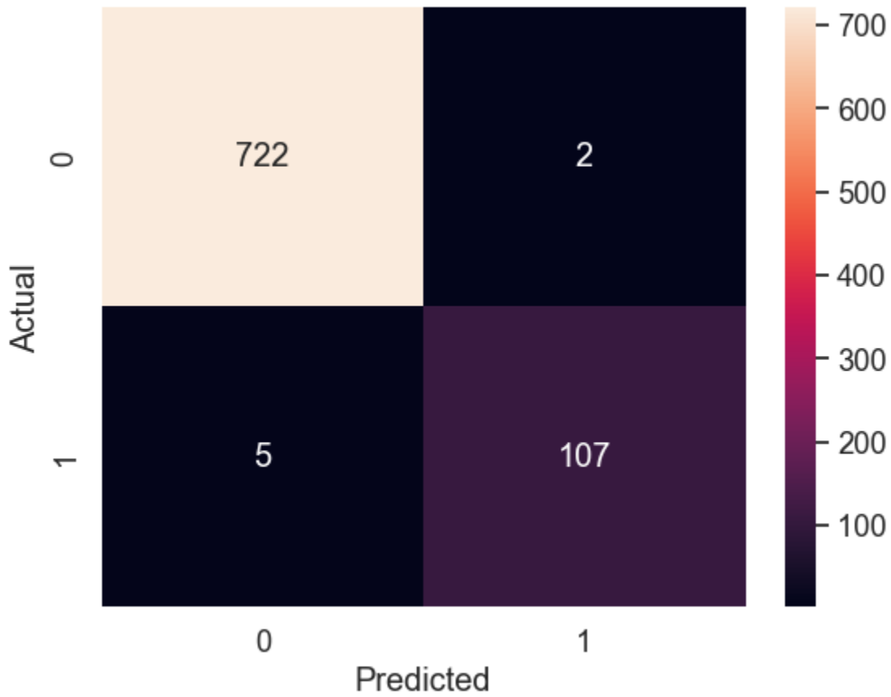
Out[40]:
```
array([0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0,
       0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1])
```

In [41]:
```
cm_hp = confusion_matrix(y_test, y_predict_hp)
cm_hp
```

Out[41]:
```
array([[722,   2],
       [  5, 107]], dtype=int64)
```

In [42]:
```
sns.heatmap(cm_hp, annot=True, fmt = 'd')
plt.xlabel('Predicted')
plt.ylabel('Actual')
```

Out[42]:
```
Text(43.25, 0.5, 'Actual')
```



In [43]:
```
nml_hp = classification_report(y_test, y_predict_hp, target_names=['Ham', 'Spam'])
print(nml_hp)
```

```
              precision    recall  f1-score   support

         Ham       0.99      1.00      1.00       724
        Spam       0.98      0.96      0.97       112

    accuracy                           0.99       836
   macro avg       0.99      0.98      0.98       836
weighted avg       0.99      0.99      0.99       836
```

Additional Metrics (ROC-AUC) for BERT

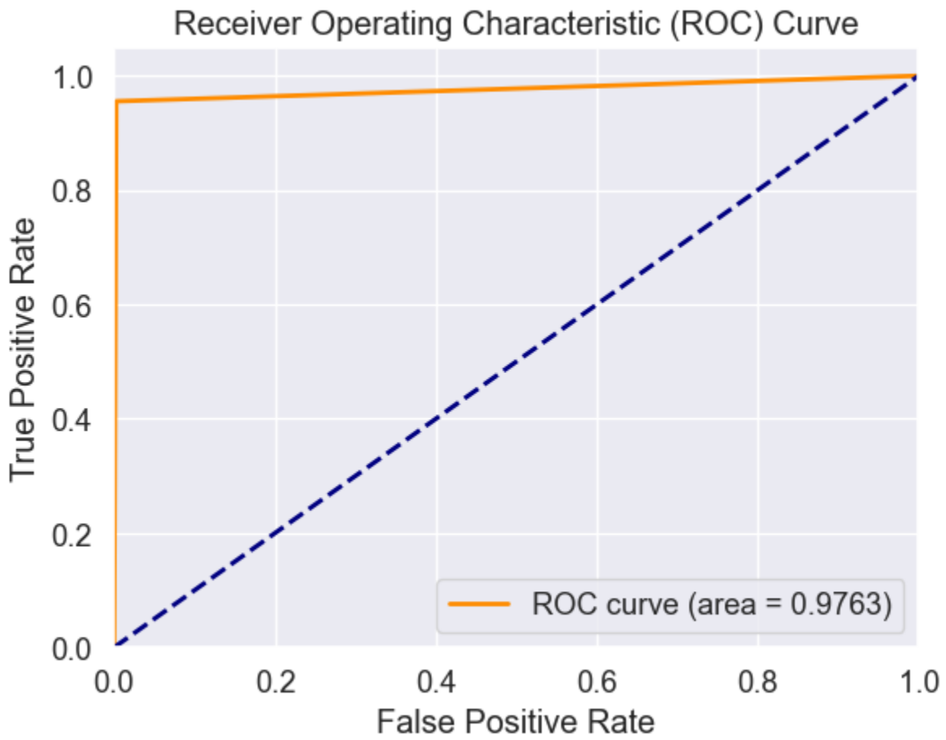
Table of Contents

In [44]:
```
from sklearn.metrics import roc_auc_score

roc_auc_test = roc_auc_score(y_test, y_predict_hp)
print(f'ROC-AUC Score on Test Set: {roc_auc_test:.4f}')
```

```
ROC-AUC Score on Test Set: 0.9763
```

In [45]:
```
from sklearn.metrics import roc_curve, auc

# Assuming y_test and y_predict_hp are already defined
fpr, tpr, _ = roc_curve(y_test, y_predict_hp)
roc_auc = auc(fpr, tpr)

# Plotting the ROC curve
plt.figure()
```

```
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()
```



In [46]:
```
# Detection of examples
nml_inf_hp = best_model_nml.predict(reviews)
nml_inf_hp
```

```
1/1 [==============================] - 1s 743ms/step
```

Out[46]:
```
array([[0.02125646],
       [0.49479303],
       [0.3908384 ],
       [0.3639348 ],
       [0.00291554]], dtype=float32)
```

Downsampling Data

Table of Contents

In [47]:
```
data_dwn = data.copy(deep=True)
```

In [48]:
```
data_dwn.head()
```

Out[48]:

	Class	Text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

In [49]:
```
data_spam = data_dwn[data_dwn['Class']==1]
data_spam.shape
```

Out[49]:
```
(747, 2)
```

In [50]:
```
data_ham = data_dwn[data_dwn['Class']==0]
data_ham.shape
```

Out[50]:
```
(4825, 2)
```

In [51]:
```
data_ham_downsampled = data_ham.sample(data_spam.shape[0])
data_ham_downsampled.shape
```

Out[51]:
```
(747, 2)
```

In [52]:
```
data_balanced_dwn = pd.concat([data_spam,data_ham_downsampled])
data_balanced_dwn.shape
```

Out[52]:
```
(1494, 2)
```

In [53]:
```
data_balanced_dwn['Class'].value_counts()
```

Out[53]:
```
Class
1    747
0    747
Name: count, dtype: int64
```

Train/Test Splitting

Table of Contents

In [54]:
```
X_dwn = data_balanced_dwn['Text']
y_dwn = data_balanced_dwn['Class']
```

In [55]:
```
# Split data into training and temporary sets (60% train, 40% temp)
X_train_dwn, X_temp_dwn, y_train_dwn, y_temp_dwn = train_test_split(X_dwn, y_dwn, test_size=0.3, random_state=random_seed, stratify = y_dwn)

# Split temporary set into validation and test sets (20% val, 20% test)
X_val_dwn, X_test_dwn, y_val_dwn, y_test_dwn = train_test_split(X_temp_dwn, y_temp_dwn, test_size=0.5, random_state=random_seed, stratify = y_temp_dwn)

print(f'Training data: {len(X_train_dwn)}, {len(y_train_dwn)}')
print(f'Validation data: {len(X_val_dwn)}, {len(y_val_dwn)}')
print(f'Testing data: {len(X_test_dwn)}, {len(y_test_dwn)}')
```

```
Training data: 1045, 1045
Validation data: 224, 224
Testing data: 225, 225
```

Model Building

Table of Contents

In [56]:
```
%%time
# Fit the model
history_dwn = model.fit(X_train_dwn, y_train_dwn, epochs=20,
                    validation_data=(X_val_dwn, y_val_dwn),
                    callbacks=[early_stopping, lr_scheduler])
```

```
Epoch 1/20
33/33 [==============================] - 17s 527ms/step - loss: 0.6254 - accuracy: 0.8000 - precision: 1.0000 - recall: 0.6004 - val_loss: 0.0899 - val_accuracy: 0.9598 - val_precision: 1.0000
- val_recall: 0.9196 - lr: 2.0000e-04
Epoch 2/20
33/33 [==============================] - 17s 511ms/step - loss: 0.3235 - accuracy: 0.8928 - precision: 0.9928 - recall: 0.7916 - val_loss: 0.1229 - val_accuracy: 0.9509 - val_precision: 1.0000
- val_recall: 0.9018 - lr: 2.0000e-04
Epoch 3/20
33/33 [==============================] - 17s 511ms/step - loss: 0.2516 - accuracy: 0.9244 - precision: 0.9868 - recall: 0.8604 - val_loss: 0.0659 - val_accuracy: 0.9598 - val_precision: 0.9402
- val_recall: 0.9821 - lr: 2.0000e-04
Epoch 4/20
33/33 [==============================] - 17s 511ms/step - loss: 0.2154 - accuracy: 0.9368 - precision: 0.9751 - recall: 0.8967 - val_loss: 0.0378 - val_accuracy: 0.9866 - val_precision: 1.0000
- val_recall: 0.9732 - lr: 2.0000e-04
Epoch 5/20
33/33 [==============================] - 17s 504ms/step - loss: 0.2313 - accuracy: 0.9407 - precision: 0.9733 - recall: 0.9063 - val_loss: 0.0633 - val_accuracy: 0.9777 - val_precision: 0.9652
- val_recall: 0.9911 - lr: 2.0000e-04
Epoch 6/20
33/33 [==============================] - 17s 504ms/step - loss: 0.1786 - accuracy: 0.9483 - precision: 0.9681 - recall: 0.9273 - val_loss: 0.0407 - val_accuracy: 0.9866 - val_precision: 1.0000
- val_recall: 0.9732 - lr: 2.0000e-04
Epoch 7/20
33/33 [==============================] - 17s 508ms/step - loss: 0.1841 - accuracy: 0.9445 - precision: 0.9568 - recall: 0.9312 - val_loss: 0.0434 - val_accuracy: 0.9866 - val_precision: 1.0000
- val_recall: 0.9732 - lr: 4.0000e-05
CPU times: total: 1min 25s
Wall time: 1min 57s
```

Model Evaluation

Table of Contents

In [57]:
```
# Evaluate the model
results_dwn = model.evaluate(X_test_dwn, y_test_dwn)
print(f"Test Loss: {results_dwn[0]}")
print(f"Test Accuracy: {results_dwn[1]}")
```

```
8/8 [==============================] - 3s 369ms/step - loss: 0.1051 - accuracy: 0.9778 - precision: 0.9908 - recall: 0.9643
Test Loss: 0.10508234798908234
Test Accuracy: 0.9777777791023254
```

In [58]:
```
y_predict_dwn = model.predict(X_test_dwn)
y_predict_dwn = y_predict_dwn.flatten()
```

```
8/8 [==============================] - 3s 402ms/step
```

In [59]:
```
y_predict_dwn = np.where(y_predict_dwn > 0.5,1,0)
y_predict_dwn
```

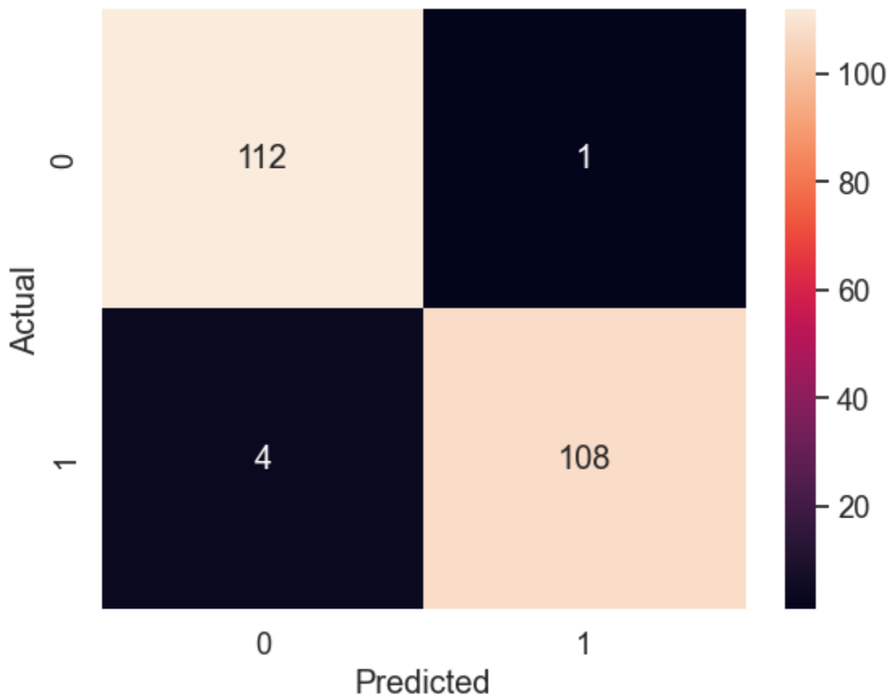
Out[59]:
```
array([1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1,
       1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0,
       1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1,
       1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 1,
       1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1,
       1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0,
       1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1,
       0, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1,
       1, 0, 1, 0, 1])
```

In [60]:
```
cm_dwn = confusion_matrix(y_test_dwn, y_predict_dwn)
cm_dwn
```

Out[60]:
```
array([[112,   1],
       [  4, 108]], dtype=int64)
```

In [61]:
```
sns.heatmap(cm_dwn, annot=True, fmt = 'd')
plt.xlabel('Predicted')
plt.ylabel('Actual')
```

Out[61]:
```
Text(43.25, 0.5, 'Actual')
```



In [62]:
```
dwn = classification_report(y_test_dwn, y_predict_dwn, target_names=['Ham', 'Spam'])
print(dwn)
```

```
              precision    recall  f1-score   support

         Ham       0.97      0.99      0.98       113
        Spam       0.99      0.96      0.98       112

    accuracy                           0.98       225
   macro avg       0.98      0.98      0.98       225
weighted avg       0.98      0.98      0.98       225
```

In [63]:
```python
# Detection of examples
dwn_inf = model.predict(reviews)
dwn_inf
```

```
1/1 [==============================] - 0s 57ms/step
```

Out[63]:
```
array([[1.5257216e-02],
       [7.4222004e-01],
       [8.9507455e-01],
       [8.9537144e-01],
       [1.8187187e-04]], dtype=float32)
```

Hyperparameter Tuning with Keras Tuner (Downsampled Data)

Table of Contents

In [64]:
```python
%%time
# Initialize Keras Tuner with the existing model
tuner = kt.Hyperband(
    compile_model,
    objective='val_accuracy',
    max_epochs=5,
    factor=3,
    directory=os.getcwd(),
    project_name='up_model'
)

# Perform hyperparameter tuning
tuner.search(X_train_dwn, y_train_dwn, epochs=5, validation_data=(X_val_dwn, y_val_dwn))

# Get the best model
best_model_dwn = tuner.get_best_models(num_models=1)[0]
```

```
Trial 3 Complete [00h 00m 41s]
val_accuracy: 0.96875

Best val_accuracy So Far: 0.9910714030265808
Total elapsed time: 00h 01m 58s
CPU times: total: 1min 30s
Wall time: 1min 59s
```

In [65]:
```python
# Evaluate the best model on the validation set
hp_results_val_dwn = best_model_dwn.evaluate(X_val_dwn, y_val_dwn)
print(f"\nValidation Loss: {hp_results_val[0]}")
print(f"Validation Accuracy: {hp_results_val[1]}")
```

```
7/7 [==============================] - 4s 413ms/step - loss: 0.0381 - accuracy: 0.9799 - precision: 1.0000 - recall: 0.9598

Validation Loss: 0.04315700754523277
Validation Accuracy: 0.9880383014678955
```

In [66]:
```
# Evaluate the best model on the Test set
hp_results_val_dwn = best_model_dwn.evaluate(X_test_dwn, y_test_dwn)
print(f"\Test Loss: {hp_results_val[0]}")
print(f"Test Accuracy: {hp_results_val[1]}")
```

```
8/8 [==============================] - 4s 362ms/step - loss: 0.1020 - accuracy: 0.9733 - precision: 0.9818 - recall: 0.9643
\Test Loss: 0.04315700754523277
Test Accuracy: 0.9880383014678955
```

Model Building

Table of Contents

In [67]:
```
%%time
# Fitting best model on X_train, y_train, X_val, y_val, X_test, y_test
best_model_dwn.fit(X_train_dwn, y_train_dwn, epochs=20,
                   validation_data=(X_val_dwn, y_val_dwn),
                   callbacks=[early_stopping, lr_scheduler])
```

```
Epoch 1/20
33/33 [==============================] - 20s 514ms/step - loss: 0.2194 - accuracy: 0.9359 - precision: 0.9730 - recall: 0.8967 - val_loss: 0.0387 - val_accuracy: 0.9911 - val_precision: 1.0000
- val_recall: 0.9821 - lr: 1.0000e-04
Epoch 2/20
33/33 [==============================] - 17s 503ms/step - loss: 0.1814 - accuracy: 0.9502 - precision: 0.9645 - recall: 0.9350 - val_loss: 0.0494 - val_accuracy: 0.9821 - val_precision: 0.9821
- val_recall: 0.9821 - lr: 1.0000e-04
Epoch 3/20
33/33 [==============================] - 17s 511ms/step - loss: 0.1639 - accuracy: 0.9531 - precision: 0.9702 - recall: 0.9350 - val_loss: 0.0445 - val_accuracy: 0.9911 - val_precision: 1.0000
- val_recall: 0.9821 - lr: 1.0000e-04
Epoch 4/20
33/33 [==============================] - 17s 515ms/step - loss: 0.1845 - accuracy: 0.9455 - precision: 0.9698 - recall: 0.9197 - val_loss: 0.0500 - val_accuracy: 0.9911 - val_precision: 1.0000
- val_recall: 0.9821 - lr: 2.0000e-05
CPU times: total: 51.7 s
Wall time: 1min 9s
```

Out[67]:
```
<keras.callbacks.History at 0x219eb8b1000>
```

Model Evaluation

Table of Contents

In [68]:
```
y_predict_hp_dwn = best_model_dwn.predict(X_test_dwn)
y_predict_hp_dwn = y_predict_hp_dwn.flatten()
```

```
8/8 [==============================] - 4s 400ms/step
```

In [69]:
```
y_predict_hp_dwn = np.where(y_predict_hp_dwn > 0.5,1,0)
y_predict_hp_dwn
```

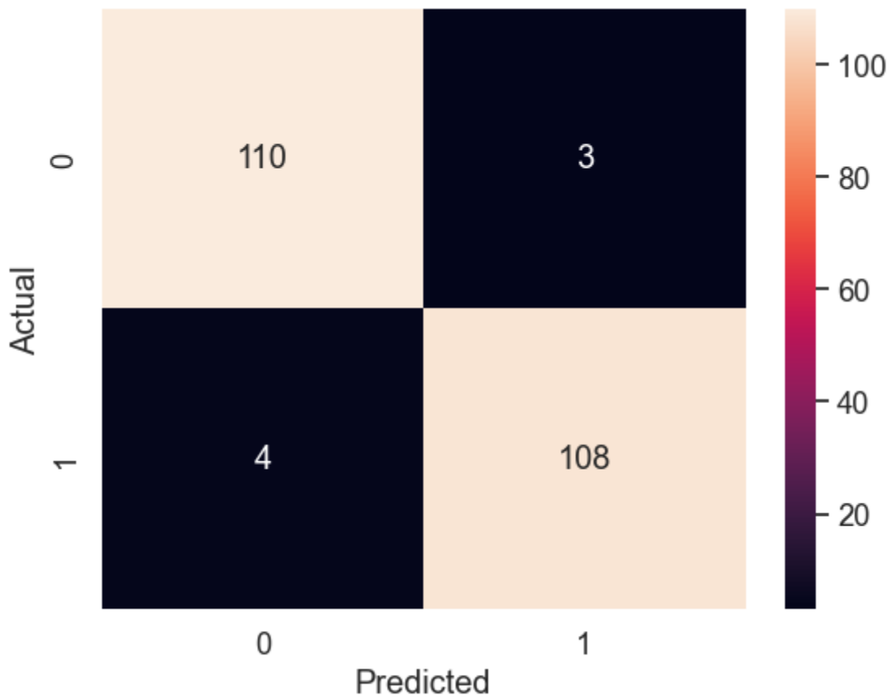
Out[69]:
```
array([1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1,
       1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0,
       1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0,
       1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1,
       1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 1,
       1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1,
       1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0,
       1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1,
       0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1,
       1, 0, 1, 0, 1])
```

In [70]:
```
cm_hp_dwn = confusion_matrix(y_test_dwn, y_predict_hp_dwn)
cm_hp_dwn
```

Out[70]:
```
array([[110,   3],
       [  4, 108]], dtype=int64)
```

In [71]:
```
sns.heatmap(cm_hp_dwn, annot=True, fmt = 'd')
plt.xlabel('Predicted')
plt.ylabel('Actual')
```

Out[71]:
```
Text(43.25, 0.5, 'Actual')
```



In [72]:
```
dwn_hp = classification_report(y_test_dwn, y_predict_hp_dwn, target_names=['Ham', 'Spam'])
print(dwn_hp)
```

```
              precision    recall  f1-score   support

         Ham       0.96      0.97      0.97       113
        Spam       0.97      0.96      0.97       112

    accuracy                           0.97       225
   macro avg       0.97      0.97      0.97       225
weighted avg       0.97      0.97      0.97       225
```

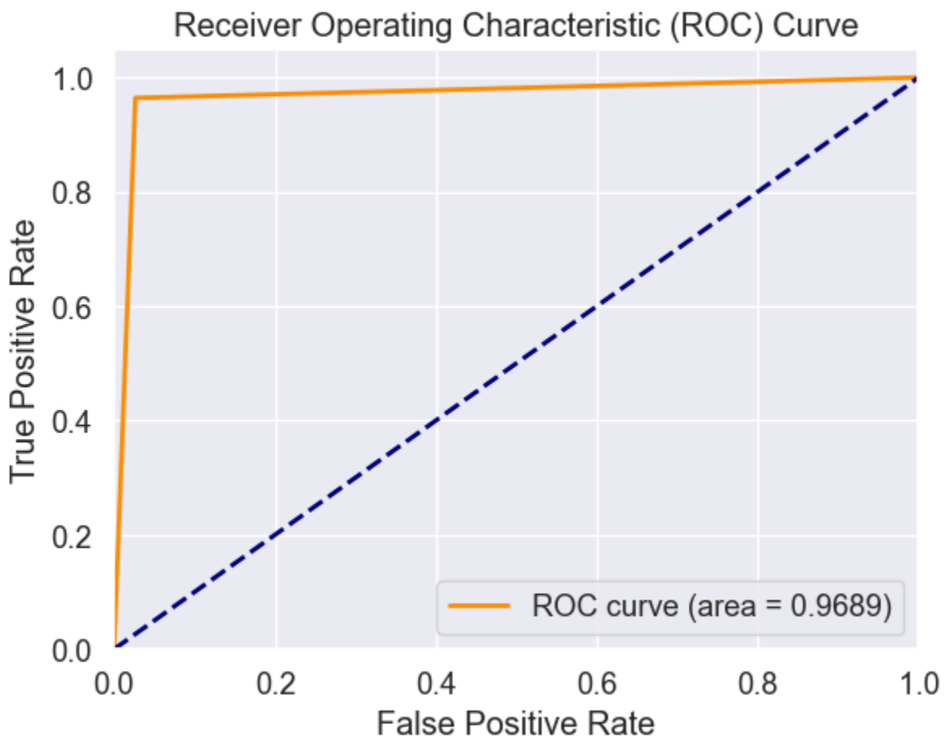
Additional Metrics (ROC-AUC) for BERT

Table of Contents

In [73]:
```
roc_auc_test = roc_auc_score(y_test_dwn, y_predict_hp_dwn)
print(f'ROC-AUC Score on Test Set: {roc_auc_test:.4f}')
```

```
ROC-AUC Score on Test Set: 0.9689
```

In [74]:
```
# y_test and y_predict_hp are already defined
fpr, tpr, _ = roc_curve(y_test_dwn, y_predict_hp_dwn)
roc_auc = auc(fpr, tpr)

# Plotting the ROC curve
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()
```



In [75]:
```
# Detection of examples
dwn_inf_hp = best_model_dwn.predict(reviews)
dwn_inf_hp
```

```
1/1 [==============================] - 1s 745ms/step
```

Out[75]:
```
array([[4.1261043e-02],
       [7.5921613e-01],
       [9.5667255e-01],
       [9.2019540e-01],
       [2.1664958e-04]], dtype=float32)
```

Upsampling Data

Table of Contents

In [76]:
```
data_up = data.copy(deep=True)
data_up
```

Out[76]:

	Class	Text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...
...	...	...
5567	1	This is the 2nd time we have tried 2 contact u...
5568	0	Will Ì_ b going to esplanade fr home?
5569	0	Pity, * was in mood for that. So...any other s...
5570	0	The guy did some bitching but I acted like i'd...
5571	0	Rofl. Its true to its name

5572 rows × 2 columns

In [77]:
```
data_spam_up = data_up[data_up['Class']==1]
data_spam_up.shape
```

Out[77]:
```
(747, 2)
```

In [78]:
```
data_ham_up = data_up[data_up['Class']==0]
data_ham_up.shape
```

Out[78]:
```
(4825, 2)
```

In [79]:
```
data_spam_upsampled = data_spam_up.sample(data_ham_up.shape[0], replace=True)
data_spam_upsampled.shape
```

Out[79]:
```
(4825, 2)
```

In [80]:
```
data_balanced_up = pd.concat([data_ham_up,data_spam_upsampled])
data_balanced_up.shape
```

Out[80]:
```
(9650, 2)
```

In [81]:
```
data_balanced_up['Class'].value_counts()
```

Out[81]:
```
Class
0    4825
1    4825
Name: count, dtype: int64
```

Train/Test Splitting

Table of Contents

In [82]:
```
X_up = data_balanced_up['Text']
```

```
y_up = data_balanced_up['Class']
```

X_train_up, X_test_up, y_train_up, y_test_up = train_test_split(X_up, y_up, test_size = 0.3, random_state=random_seed, stratify = y_up) print(f'Training data: {len(X_train_up)}, {len(y_train_up)}') print(f'Testing data: {len(X_test_up)}, {len(y_test_up)}')

In [83]:
```
# Split data into training and temporary sets (60% train, 40% temp)
X_train_up, X_temp_up, y_train_up, y_temp_up = train_test_split(X_up, y_up, test_size=0.3, random_state=random_seed, stratify = y_up)

# Split temporary set into validation and test sets (20% val, 20% test)
X_val_up, X_test_up, y_val_up, y_test_up = train_test_split(X_temp_up, y_temp_up, test_size=0.5, random_state=random_seed, stratify = y_temp_up)

print(f'Training data: {len(X_train_up)}, {len(y_train_up)}')
print(f'Validation data: {len(X_val_up)}, {len(y_val_up)}')
print(f'Testing data: {len(X_test_up)}, {len(y_test_up)}')
```

```
Training data: 6755, 6755
Validation data: 1447, 1447
Testing data: 1448, 1448
```

Model Building

Table of Contents

In [84]:
```
%%time
# Fit the model
history_up = model.fit(X_train_up, y_train_up, epochs=20,
                    validation_data=(X_val_up, y_val_up),
                    callbacks=[early_stopping, lr_scheduler])
```

```
Epoch 1/20
212/212 [==============================] - 109s 515ms/step - loss: 0.1921 - accuracy: 0.9466 - precision: 0.9733 - recall: 0.9183 - val_loss: 0.1202 - val_accuracy: 0.9758 - val_precision: 0.9
831 - val_recall: 0.9682 - lr: 2.0000e-05
Epoch 2/20
212/212 [==============================] - 109s 513ms/step - loss: 0.1866 - accuracy: 0.9470 - precision: 0.9731 - recall: 0.9195 - val_loss: 0.1292 - val_accuracy: 0.9793 - val_precision: 0.9
901 - val_recall: 0.9682 - lr: 2.0000e-05
Epoch 3/20
212/212 [==============================] - 109s 514ms/step - loss: 0.1799 - accuracy: 0.9495 - precision: 0.9700 - recall: 0.9278 - val_loss: 0.1198 - val_accuracy: 0.9765 - val_precision: 0.9
845 - val_recall: 0.9682 - lr: 2.0000e-05
Epoch 4/20
212/212 [==============================] - 108s 511ms/step - loss: 0.1731 - accuracy: 0.9531 - precision: 0.9690 - recall: 0.9361 - val_loss: 0.1282 - val_accuracy: 0.9800 - val_precision: 0.9
929 - val_recall: 0.9668 - lr: 2.0000e-05
Epoch 5/20
212/212 [==============================] - 107s 507ms/step - loss: 0.1779 - accuracy: 0.9522 - precision: 0.9670 - recall: 0.9364 - val_loss: 0.1162 - val_accuracy: 0.9772 - val_precision: 0.9
859 - val_recall: 0.9682 - lr: 2.0000e-05
Epoch 6/20
212/212 [==============================] - 108s 508ms/step - loss: 0.1811 - accuracy: 0.9523 - precision: 0.9719 - recall: 0.9316 - val_loss: 0.1158 - val_accuracy: 0.9779 - val_precision: 0.9
873 - val_recall: 0.9682 - lr: 2.0000e-05
Epoch 7/20
212/212 [==============================] - 108s 510ms/step - loss: 0.1583 - accuracy: 0.9556 - precision: 0.9644 - recall: 0.9461 - val_loss: 0.1173 - val_accuracy: 0.9793 - val_precision: 0.9
901 - val_recall: 0.9682 - lr: 2.0000e-05
Epoch 8/20
212/212 [==============================] - 108s 511ms/step - loss: 0.1666 - accuracy: 0.9531 - precision: 0.9651 - recall: 0.9402 - val_loss: 0.1095 - val_accuracy: 0.9772 - val_precision: 0.9
859 - val_recall: 0.9682 - lr: 2.0000e-05
Epoch 9/20
212/212 [==============================] - 108s 509ms/step - loss: 0.1535 - accuracy: 0.9565 - precision: 0.9678 - recall: 0.9443 - val_loss: 0.1071 - val_accuracy: 0.9772 - val_precision: 0.9
859 - val_recall: 0.9682 - lr: 2.0000e-05
Epoch 10/20
212/212 [==============================] - 109s 513ms/step - loss: 0.1501 - accuracy: 0.9571 - precision: 0.9682 - recall: 0.9452 - val_loss: 0.1065 - val_accuracy: 0.9772 - val_precision: 0.9
859 - val_recall: 0.9682 - lr: 2.0000e-05
Epoch 11/20
212/212 [==============================] - 108s 511ms/step - loss: 0.1538 - accuracy: 0.9574 - precision: 0.9651 - recall: 0.9491 - val_loss: 0.1018 - val_accuracy: 0.9772 - val_precision: 0.9
859 - val_recall: 0.9682 - lr: 2.0000e-05
Epoch 12/20
212/212 [==============================] - 108s 512ms/step - loss: 0.1533 - accuracy: 0.9581 - precision: 0.9668 - recall: 0.9488 - val_loss: 0.0952 - val_accuracy: 0.9758 - val_precision: 0.9
804 - val_recall: 0.9710 - lr: 2.0000e-05
Epoch 13/20
212/212 [==============================] - 108s 510ms/step - loss: 0.1546 - accuracy: 0.9535 - precision: 0.9637 - recall: 0.9426 - val_loss: 0.0990 - val_accuracy: 0.9772 - val_precision: 0.9
859 - val_recall: 0.9682 - lr: 2.0000e-05
Epoch 14/20
212/212 [==============================] - 108s 509ms/step - loss: 0.1456 - accuracy: 0.9580 - precision: 0.9662 - recall: 0.9491 - val_loss: 0.0952 - val_accuracy: 0.9793 - val_precision: 0.9
873 - val_recall: 0.9710 - lr: 2.0000e-05
Epoch 15/20
212/212 [==============================] - 107s 507ms/step - loss: 0.1488 - accuracy: 0.9565 - precision: 0.9625 - recall: 0.9500 - val_loss: 0.0939 - val_accuracy: 0.9793 - val_precision: 0.9
873 - val_recall: 0.9710 - lr: 4.0000e-06
Epoch 16/20
212/212 [==============================] - 107s 506ms/step - loss: 0.1544 - accuracy: 0.9544 - precision: 0.9623 - recall: 0.9458 - val_loss: 0.0942 - val_accuracy: 0.9793 - val_precision: 0.9
873 - val_recall: 0.9710 - lr: 4.0000e-06
Epoch 17/20
212/212 [==============================] - 107s 507ms/step - loss: 0.1381 - accuracy: 0.9605 - precision: 0.9653 - recall: 0.9553 - val_loss: 0.0948 - val_accuracy: 0.9779 - val_precision: 0.9
873 - val_recall: 0.9682 - lr: 4.0000e-06
Epoch 18/20
212/212 [==============================] - 107s 507ms/step - loss: 0.1375 - accuracy: 0.9594 - precision: 0.9675 - recall: 0.9509 - val_loss: 0.0926 - val_accuracy: 0.9779 - val_precision: 0.9
846 - val_recall: 0.9710 - lr: 8.0000e-07
Epoch 19/20
212/212 [==============================] - 108s 509ms/step - loss: 0.1429 - accuracy: 0.9577 - precision: 0.9626 - recall: 0.9523 - val_loss: 0.0927 - val_accuracy: 0.9793 - val_precision: 0.9
873 - val_recall: 0.9710 - lr: 8.0000e-07
Epoch 20/20
212/212 [==============================] - 107s 506ms/step - loss: 0.1449 - accuracy: 0.9553 - precision: 0.9624 - recall: 0.9476 - val_loss: 0.0940 - val_accuracy: 0.9793 - val_precision: 0.9
873 - val_recall: 0.9710 - lr: 8.0000e-07
```

```
CPU times: total: 26min 11s
Wall time: 35min 59s
```

Model Evaluation

Table of Contents

In [85]:
```
# Evaluate the model
results_up = model.evaluate(X_test_up, y_test_up)
print(f"Test Loss: {results_up[0]}")
print(f"Test Accuracy: {results_up[1]}")
```

```
46/46 [==============================] - 19s 409ms/step - loss: 0.0941 - accuracy: 0.9738 - precision: 0.9777 - recall: 0.9696
Test Loss: 0.09409315139055252
Test Accuracy: 0.9737569093704224
```

In [86]:
```
y_predict_up = model.predict(X_test_up)
y_predict_up = y_predict_up.flatten()
```

```
46/46 [==============================] - 19s 411ms/step
```

In [87]:
```
y_predict_up = np.where(y_predict_up > 0.5,1,0)
y_predict_up
```

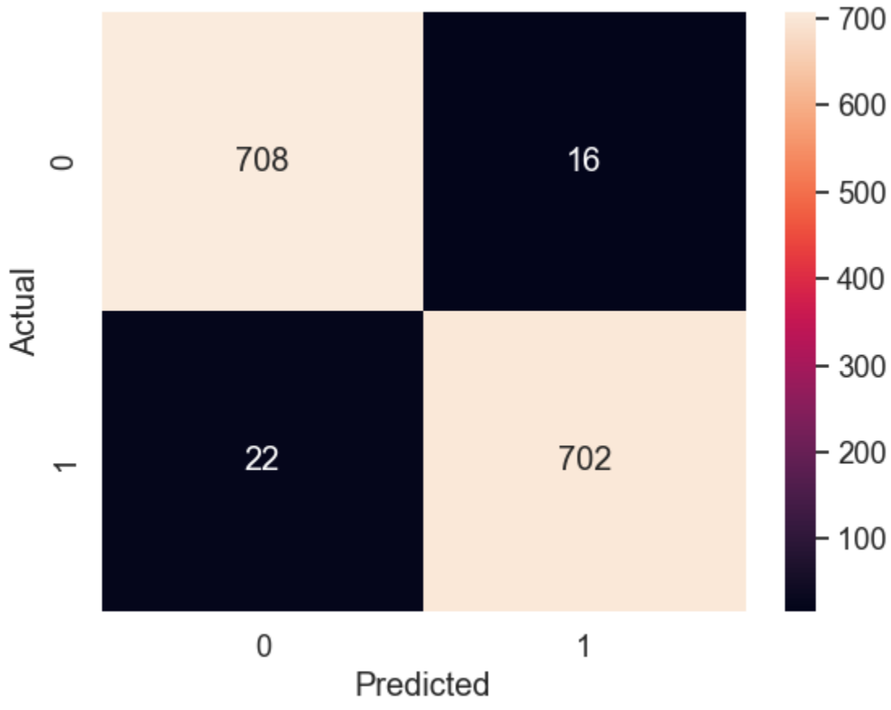
Out[87]:
```
array([1, 1, 1, ..., 1, 0, 1])
```

In [88]:
```
cm_up = confusion_matrix(y_test_up, y_predict_up)
cm_up
```

Out[88]:
```
array([[708,  16],
       [ 22, 702]], dtype=int64)
```

In [89]:
```
sns.heatmap(cm_up, annot=True, fmt = 'd')
plt.xlabel('Predicted')
plt.ylabel('Actual')
```

Out[89]:
```
Text(43.25, 0.5, 'Actual')
```



In [90]:
```
up = classification_report(y_test_up, y_predict_up, target_names=['Ham', 'Spam'])
print(up)
```

```
              precision    recall  f1-score   support

         Ham       0.97      0.98      0.97       724
        Spam       0.98      0.97      0.97       724

    accuracy                           0.97      1448
   macro avg       0.97      0.97      0.97      1448
weighted avg       0.97      0.97      0.97      1448
```

In [91]:
```
# Detection of examples
up_inf = model.predict(reviews)
up_inf
```

```
1/1 [==============================] - 0s 49ms/step
```

Out[91]:
```
array([[8.3901323e-02],
       [7.4371928e-01],
       [8.2814676e-01],
       [8.7304598e-01],
       [2.8097225e-05]], dtype=float32)
```

Hyperparameter Tuning with Keras Tuner (Upsampled Data)

Table of Contents

In [92]:
```
%%time
# Initialize Keras Tuner with the existing model
tuner = kt.Hyperband(
```

```
    compile_model,
    objective='val_accuracy',
    max_epochs=5,
    factor=3,
    directory=os.getcwd(),
    project_name='nml_model'
)

# Perform hyperparameter tuning
tuner.search(X_train_up, y_train_up, epochs=5, validation_data=(X_val_up, y_val_up))

# Get the best model
best_model_up = tuner.get_best_models(num_models=1)[0]
```

```
Reloading Tuner from C:\Users\sathi\Career\Internship\AFAME\Spam SMS Detection\BERT\nml_model\tuner0.json
CPU times: total: 1.27 s
Wall time: 1.61 s
```

In [93]:
```
# Evaluate the best model on the validation set
hp_results_val_up = best_model_up.evaluate(X_val_up, y_val_up)
print(f"\nValidation Loss: {hp_results_val_up[0]}")
print(f"Validation Accuracy: {hp_results_val_up[1]}")
```

```
46/46 [==============================] - 20s 408ms/step - loss: 0.0920 - accuracy: 0.9710 - precision: 0.9837 - recall: 0.9578

Validation Loss: 0.09195586293935776
Validation Accuracy: 0.9709844589233398
```

In [94]:
```
# Evaluate the best model on the Test set
hp_results_val_up = best_model_up.evaluate(X_test_up, y_test_up)
print(f"\Test Loss: {hp_results_val_up[0]}")
print(f"Test Accuracy: {hp_results_val_up[1]}")
```

```
46/46 [==============================] - 19s 410ms/step - loss: 0.1000 - accuracy: 0.9620 - precision: 0.9898 - recall: 0.9337
\Test Loss: 0.09998809546232224
Test Accuracy: 0.9620165824890137
```

Model Building

Table of Contents

In [95]:
```
%%time
# Fitting best model on X_train, y_train, X_val, y_val, X_test, y_test
best_model_up.fit(X_train_up, y_train_up, epochs=20,
                  validation_data=(X_val_up, y_val_up),
                  callbacks=[early_stopping, lr_scheduler])
```

```
Epoch 1/20
212/212 [==============================] - 113s 518ms/step - loss: 0.2159 - accuracy: 0.9341 - precision: 0.9579 - recall: 0.9082 - val_loss: 0.1302 - val_accuracy: 0.9696 - val_precision: 0.9
971 - val_recall: 0.9419 - lr: 0.0010
Epoch 2/20
212/212 [==============================] - 107s 507ms/step - loss: 0.1412 - accuracy: 0.9510 - precision: 0.9519 - recall: 0.9500 - val_loss: 0.0725 - val_accuracy: 0.9841 - val_precision: 0.9
944 - val_recall: 0.9737 - lr: 0.0010
Epoch 3/20
212/212 [==============================] - 108s 509ms/step - loss: 0.1287 - accuracy: 0.9538 - precision: 0.9538 - recall: 0.9538 - val_loss: 0.0935 - val_accuracy: 0.9724 - val_precision: 0.9
971 - val_recall: 0.9474 - lr: 0.0010
Epoch 4/20
212/212 [==============================] - 107s 506ms/step - loss: 0.1178 - accuracy: 0.9559 - precision: 0.9583 - recall: 0.9532 - val_loss: 0.4069 - val_accuracy: 0.8701 - val_precision: 1.0
000 - val_recall: 0.7400 - lr: 0.0010
Epoch 5/20
212/212 [==============================] - 107s 507ms/step - loss: 0.1006 - accuracy: 0.9637 - precision: 0.9666 - recall: 0.9606 - val_loss: 0.0561 - val_accuracy: 0.9834 - val_precision: 0.9
916 - val_recall: 0.9751 - lr: 2.0000e-04
Epoch 6/20
212/212 [==============================] - 107s 506ms/step - loss: 0.1055 - accuracy: 0.9614 - precision: 0.9623 - recall: 0.9603 - val_loss: 0.0591 - val_accuracy: 0.9834 - val_precision: 0.9
888 - val_recall: 0.9779 - lr: 2.0000e-04
Epoch 7/20
212/212 [==============================] - 107s 507ms/step - loss: 0.0952 - accuracy: 0.9624 - precision: 0.9600 - recall: 0.9651 - val_loss: 0.0572 - val_accuracy: 0.9834 - val_precision: 0.9
807 - val_recall: 0.9862 - lr: 2.0000e-04
Epoch 8/20
212/212 [==============================] - 109s 513ms/step - loss: 0.0934 - accuracy: 0.9657 - precision: 0.9698 - recall: 0.9612 - val_loss: 0.0525 - val_accuracy: 0.9834 - val_precision: 0.9
916 - val_recall: 0.9751 - lr: 4.0000e-05
Epoch 9/20
212/212 [==============================] - 108s 512ms/step - loss: 0.0965 - accuracy: 0.9643 - precision: 0.9670 - recall: 0.9615 - val_loss: 0.0524 - val_accuracy: 0.9855 - val_precision: 0.9
916 - val_recall: 0.9793 - lr: 4.0000e-05
Epoch 10/20
212/212 [==============================] - 109s 512ms/step - loss: 0.0905 - accuracy: 0.9649 - precision: 0.9640 - recall: 0.9660 - val_loss: 0.0519 - val_accuracy: 0.9855 - val_precision: 0.9
916 - val_recall: 0.9793 - lr: 4.0000e-05
Epoch 11/20
212/212 [==============================] - 110s 517ms/step - loss: 0.0954 - accuracy: 0.9630 - precision: 0.9630 - recall: 0.9630 - val_loss: 0.0514 - val_accuracy: 0.9855 - val_precision: 0.9
916 - val_recall: 0.9793 - lr: 4.0000e-05
Epoch 12/20
212/212 [==============================] - 108s 511ms/step - loss: 0.0974 - accuracy: 0.9624 - precision: 0.9619 - recall: 0.9630 - val_loss: 0.0524 - val_accuracy: 0.9855 - val_precision: 0.9
916 - val_recall: 0.9793 - lr: 4.0000e-05
Epoch 13/20
212/212 [==============================] - 108s 511ms/step - loss: 0.0847 - accuracy: 0.9682 - precision: 0.9711 - recall: 0.9651 - val_loss: 0.0512 - val_accuracy: 0.9855 - val_precision: 0.9
902 - val_recall: 0.9806 - lr: 4.0000e-05
Epoch 14/20
212/212 [==============================] - 108s 512ms/step - loss: 0.0882 - accuracy: 0.9677 - precision: 0.9686 - recall: 0.9668 - val_loss: 0.0500 - val_accuracy: 0.9876 - val_precision: 0.9
916 - val_recall: 0.9834 - lr: 4.0000e-05
Epoch 15/20
212/212 [==============================] - 108s 510ms/step - loss: 0.0851 - accuracy: 0.9686 - precision: 0.9711 - recall: 0.9660 - val_loss: 0.0507 - val_accuracy: 0.9876 - val_precision: 0.9
916 - val_recall: 0.9834 - lr: 4.0000e-05
Epoch 16/20
212/212 [==============================] - 108s 512ms/step - loss: 0.0955 - accuracy: 0.9628 - precision: 0.9613 - recall: 0.9645 - val_loss: 0.0502 - val_accuracy: 0.9869 - val_precision: 0.9
916 - val_recall: 0.9820 - lr: 4.0000e-05
Epoch 17/20
212/212 [==============================] - 108s 512ms/step - loss: 0.0881 - accuracy: 0.9651 - precision: 0.9607 - recall: 0.9698 - val_loss: 0.0498 - val_accuracy: 0.9869 - val_precision: 0.9
903 - val_recall: 0.9834 - lr: 8.0000e-06
Epoch 18/20
212/212 [==============================] - 108s 512ms/step - loss: 0.0972 - accuracy: 0.9657 - precision: 0.9654 - recall: 0.9660 - val_loss: 0.0498 - val_accuracy: 0.9876 - val_precision: 0.9
916 - val_recall: 0.9834 - lr: 8.0000e-06
Epoch 19/20
212/212 [==============================] - 108s 512ms/step - loss: 0.0916 - accuracy: 0.9674 - precision: 0.9685 - recall: 0.9663 - val_loss: 0.0497 - val_accuracy: 0.9876 - val_precision: 0.9
916 - val_recall: 0.9834 - lr: 8.0000e-06
Epoch 20/20
212/212 [==============================] - 108s 510ms/step - loss: 0.0919 - accuracy: 0.9651 - precision: 0.9645 - recall: 0.9657 - val_loss: 0.0501 - val_accuracy: 0.9869 - val_precision: 0.9
916 - val_recall: 0.9820 - lr: 1.6000e-06
```

```
CPU times: total: 26min 11s
Wall time: 36min 7s
```

Out[95]: <keras.callbacks.History at 0x219ebd34910>

Model Evaluation

Table of Contents

In [96]:
```
y_predict_hp_up = best_model_up.predict(X_test_up)
y_predict_hp_up = y_predict_hp_up.flatten()
```

```
46/46 [==============================] - 19s 412ms/step
```

In [97]:
```
y_predict_hp_up = np.where(y_predict_hp_up > 0.5,1,0)
y_predict_hp_up
```

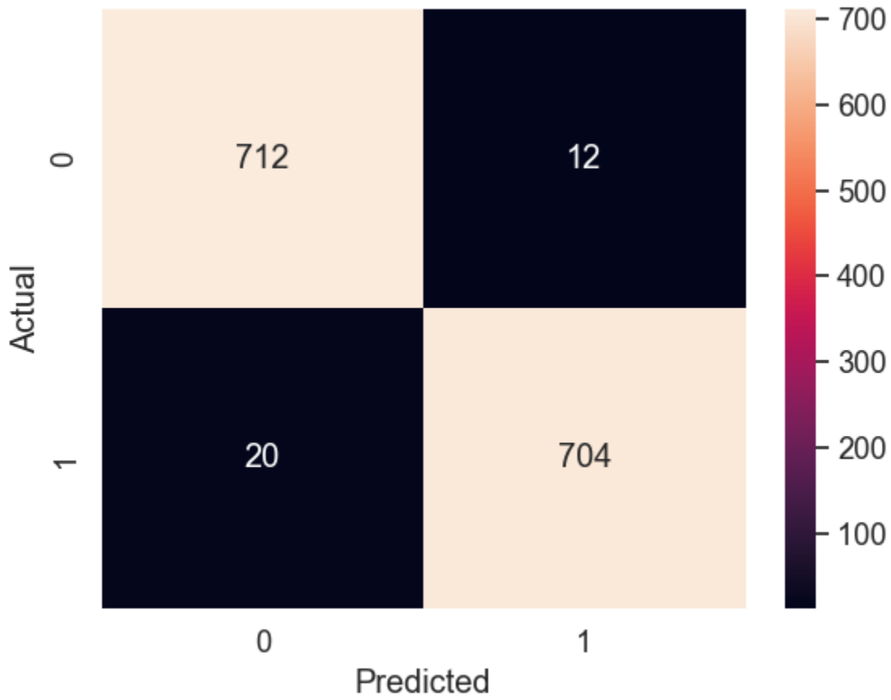
Out[97]:
```
array([1, 1, 1, ..., 1, 0, 1])
```

In [98]:
```
cm_hp_up = confusion_matrix(y_test_up, y_predict_hp_up)
cm_hp_up
```

Out[98]:
```
array([[712,  12],
       [ 20, 704]], dtype=int64)
```

In [99]:
```
sns.heatmap(cm_hp_up, annot=True, fmt = 'd')
plt.xlabel('Predicted')
plt.ylabel('Actual')
```

Out[99]:
```
Text(43.25, 0.5, 'Actual')
```



In [100…
```
up_hp = classification_report(y_test_up, y_predict_hp_up, target_names=['Ham', 'Spam'])
```

```
print(up_hp)
```

```
              precision    recall  f1-score   support

         Ham       0.97      0.98      0.98       724
        Spam       0.98      0.97      0.98       724

    accuracy                           0.98      1448
   macro avg       0.98      0.98      0.98      1448
weighted avg       0.98      0.98      0.98      1448
```

Additional Metrics (ROC-AUC) for BERT

Table of Contents

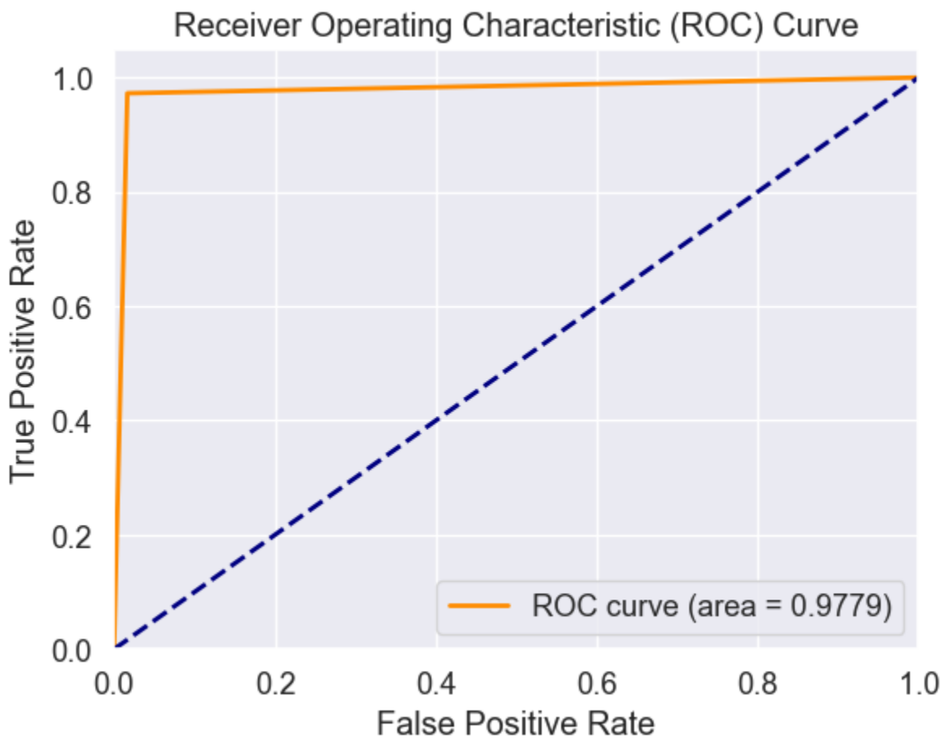
In [101…

```
roc_auc_test = roc_auc_score(y_test_up, y_predict_hp_up)
print(f'ROC-AUC Score on Test Set: {roc_auc_test:.4f}')
```

```
ROC-AUC Score on Test Set: 0.9779
```

In [102…

```
# y_test and y_predict_hp are already defined
fpr, tpr, _ = roc_curve(y_test_up, y_predict_hp_up)
roc_auc = auc(fpr, tpr)

# Plotting the ROC curve
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()
```



In [103…

```
# Detection of examples
up_inf_hp = best_model_up.predict(reviews)
up_inf_hp
```

```
1/1 [==============================] - 1s 749ms/step
```

Out[103…

```
array([[0.29685044],
       [0.8558242 ],
       [0.6424521 ],
       [0.9255771 ],
       [0.01625668]], dtype=float32)
```

Class Weights

Table of Contents

In [104…

```
# Calculate class weights
class_weights = {
    0: len(y_train) / (2 * (len(y_train) - np.sum(y_train))),  # Weight for ham
    1: len(y_train) / (2 * np.sum(y_train))  # Weight for spam
}
```

In [105…

```
%%time
clw_model = tf.keras.models.clone_model(model)
```

```
CPU times: total: 10.3 s
Wall time: 10.7 s
```

In [106…

```
# Compile the model with class weights
clw_model.compile(optimizer=Adam(learning_rate=0.001),
                  loss='binary_crossentropy',
```

```
                  metrics = metrics)
```

Model Building

Table of Contents

In [107…

```
%%time
# Fit the model
history_clw = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_val, y_val),
                    class_weight=class_weights,
                    callbacks=[early_stopping, lr_scheduler])
```

```
Epoch 1/20
122/122 [==============================] - 66s 520ms/step - loss: 0.3624 - accuracy: 0.7723 - precision: 0.3703 - recall: 0.9962 - val_loss: 0.2785 - val_accuracy: 0.8804 - val_precision: 0.52
83 - val_recall: 1.0000 - lr: 1.6000e-06
Epoch 2/20
122/122 [==============================] - 62s 512ms/step - loss: 0.3620 - accuracy: 0.7723 - precision: 0.3705 - recall: 0.9981 - val_loss: 0.4302 - val_accuracy: 0.8278 - val_precision: 0.43
75 - val_recall: 1.0000 - lr: 1.6000e-06
Epoch 3/20
122/122 [==============================] - 62s 511ms/step - loss: 0.3494 - accuracy: 0.7703 - precision: 0.3680 - recall: 0.9943 - val_loss: 0.4888 - val_accuracy: 0.8086 - val_precision: 0.41
18 - val_recall: 1.0000 - lr: 1.6000e-06
Epoch 4/20
122/122 [==============================] - 63s 514ms/step - loss: 0.3582 - accuracy: 0.7695 - precision: 0.3674 - recall: 0.9962 - val_loss: 0.5169 - val_accuracy: 0.8002 - val_precision: 0.40
14 - val_recall: 1.0000 - lr: 3.2000e-07
CPU times: total: 3min 1s
Wall time: 4min 13s
```

Model Evaluation

Table of Contents

In [108…

```
# Evaluate the model
results_clw = model.evaluate(X_test, y_test)
print(f"Test Loss: {results[0]}")
print(f"Test Accuracy: {results[1]}")
```

```
27/27 [==============================] - 11s 407ms/step - loss: 0.2016 - accuracy: 0.9175 - precision: 0.6188 - recall: 1.0000
Test Loss: 0.044614627957344055
Test Accuracy: 0.989234447479248
```

In [109…

```
y_predict_clw = clw_model.predict(X_test)
y_predict_clw = y_predict_clw.flatten()
```

```
27/27 [==============================] - 12s 417ms/step
```

In [110…

```
y_predict_clw = np.where(y_predict_clw > 0.5,1,0)
y_predict_clw
```

Out[110…

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0])
```

In [111…

```
clw_cm = confusion_matrix(y_test, y_predict)
clw_cm
```

Out[111…

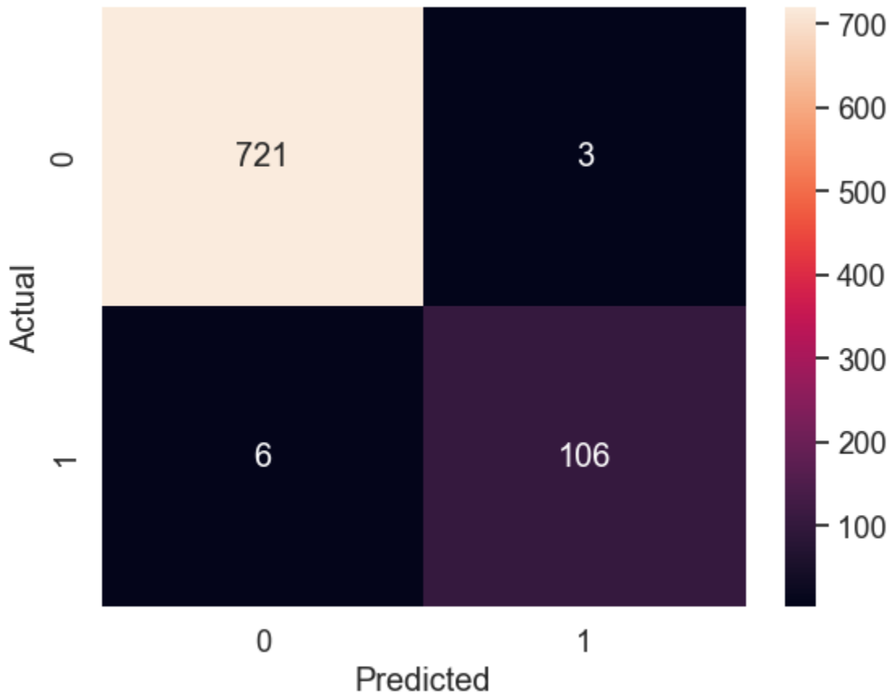
```
array([[721,   3],
       [  6, 106]], dtype=int64)
```

In [112…

```
sns.heatmap(clw_cm, annot=True, fmt = 'd')
plt.xlabel('Predicted')
plt.ylabel('Actual')
```

Out[112…

```
Text(43.25, 0.5, 'Actual')
```



In [113…

```
clw = classification_report(y_test, y_predict_clw, target_names=['Ham', 'Spam'])
print(clw)
```

```
              precision    recall  f1-score   support

         Ham       0.88      0.99      0.93       724
        Spam       0.55      0.10      0.17       112

    accuracy                           0.87       836
   macro avg       0.71      0.54      0.55       836
weighted avg       0.83      0.87      0.83       836
```

In [114…

```
# Detection of examples
clw_inf = clw_model.predict(reviews)
clw_inf
```

```
1/1 [==============================] - 1s 875ms/step
```

Out[114…

```
array([[0.48362985],
       [0.47246823],
       [0.4243976 ],
       [0.4862269 ],
       [0.47462368]], dtype=float32)
```

Hyperparameter Tuning with Keras Tuner (Class Weights)

Table of Contents

In [115…

```
%%time
# Initialize Keras Tuner with the existing model
tuner = kt.Hyperband(
```

```
    compile_model,
    objective='val_accuracy',
    max_epochs=5,
    factor=3,
    directory=os.getcwd(),
    project_name='nml_model'
)

# Perform hyperparameter tuning
tuner.search(X_train, y_train, epochs=5, validation_data=(X_val, y_val))

# Get the best model
best_model_clw = tuner.get_best_models(num_models=1)[0]
```

```
Reloading Tuner from C:\Users\sathi\Career\Internship\AFAME\Spam SMS Detection\BERT\nml_model\tuner0.json
CPU times: total: 1.62 s
Wall time: 2.15 s
```

In [116…

```
# Evaluate the best model on the validation set
hp_results_val_clw = best_model_clw.evaluate(X_val, y_val)
print(f"\nValidation Loss: {hp_results_val_clw[0]}")
print(f"Validation Accuracy: {hp_results_val_clw[1]}")
```

```
27/27 [==============================] - 12s 403ms/step - loss: 0.0516 - accuracy: 0.9492 - precision: 0.7356 - recall: 0.9688

Validation Loss: 0.05157732218503952
Validation Accuracy: 0.9491626620292664
```

In [117…

```
# Evaluate the best model on the test set
hp_results_val_clw = best_model_clw.evaluate(X_test, y_test)
print(f"\Test Loss: {hp_results_val_clw[0]}")
print(f"Test Accuracy: {hp_results_val_clw[1]}")
```

```
27/27 [==============================] - 11s 408ms/step - loss: 0.0432 - accuracy: 0.9880 - precision: 0.9554 - recall: 0.9554
\Test Loss: 0.04315700754523277
Test Accuracy: 0.9880383014678955
```

Model Building

Table of Contents

In [118…

```
%%time
# Fitting best model on X_train, y_train, X_val, y_val, X_test, y_test
best_model_clw.fit(X_train, y_train, epochs=20,
                   validation_data=(X_val, y_val),
                   callbacks=[early_stopping, lr_scheduler])
```

```
Epoch 1/20
122/122 [==============================] - 65s 514ms/step - loss: 0.0722 - accuracy: 0.9744 - precision: 0.9256 - recall: 0.8795 - val_loss: 0.0559 - val_accuracy: 0.9785 - val_precision: 0.95
19 - val_recall: 0.8839 - lr: 0.0010
Epoch 2/20
122/122 [==============================] - 155s 1s/step - loss: 0.0855 - accuracy: 0.9703 - precision: 0.9062 - recall: 0.8681 - val_loss: 0.0776 - val_accuracy: 0.9749 - val_precision: 1.0000
- val_recall: 0.8125 - lr: 0.0010
Epoch 3/20
122/122 [==============================] - 147s 1s/step - loss: 0.0734 - accuracy: 0.9738 - precision: 0.9202 - recall: 0.8815 - val_loss: 0.0580 - val_accuracy: 0.9773 - val_precision: 0.9794
- val_recall: 0.8482 - lr: 0.0010
Epoch 4/20
122/122 [==============================] - 63s 513ms/step - loss: 0.0680 - accuracy: 0.9774 - precision: 0.9341 - recall: 0.8948 - val_loss: 0.0474 - val_accuracy: 0.9821 - val_precision: 0.98
02 - val_recall: 0.8839 - lr: 2.0000e-04
Epoch 5/20
122/122 [==============================] - 63s 515ms/step - loss: 0.0707 - accuracy: 0.9782 - precision: 0.9451 - recall: 0.8891 - val_loss: 0.0474 - val_accuracy: 0.9856 - val_precision: 0.93
86 - val_recall: 0.9554 - lr: 2.0000e-04
Epoch 6/20
122/122 [==============================] - 63s 516ms/step - loss: 0.0669 - accuracy: 0.9772 - precision: 0.9289 - recall: 0.8987 - val_loss: 0.0394 - val_accuracy: 0.9880 - val_precision: 0.97
22 - val_recall: 0.9375 - lr: 2.0000e-04
Epoch 7/20
122/122 [==============================] - 63s 518ms/step - loss: 0.0595 - accuracy: 0.9803 - precision: 0.9478 - recall: 0.9025 - val_loss: 0.0409 - val_accuracy: 0.9844 - val_precision: 0.99
01 - val_recall: 0.8929 - lr: 2.0000e-04
Epoch 8/20
122/122 [==============================] - 63s 513ms/step - loss: 0.0681 - accuracy: 0.9785 - precision: 0.9312 - recall: 0.9063 - val_loss: 0.0436 - val_accuracy: 0.9844 - val_precision: 0.96
26 - val_recall: 0.9196 - lr: 2.0000e-04
Epoch 9/20
122/122 [==============================] - 62s 513ms/step - loss: 0.0656 - accuracy: 0.9782 - precision: 0.9380 - recall: 0.8967 - val_loss: 0.0424 - val_accuracy: 0.9844 - val_precision: 0.97
14 - val_recall: 0.9107 - lr: 4.0000e-05
CPU times: total: 7min 55s
Wall time: 12min 24s
```

Out[118…

```
<keras.callbacks.History at 0x219ee645570>
```

Model Evaluation

Table of Contents

In [119…

```
y_predict_hp_clw = best_model_clw.predict(X_test)
y_predict_hp_clw = y_predict_hp_clw.flatten()
```

```
27/27 [==============================] - 12s 415ms/step
```

In [120…

```
y_predict_hp_clw = np.where(y_predict_hp_clw > 0.5,1,0)
y_predict_hp_clw
```

Out[120…

```
array([0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0,
       0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1])
```

In [121…

```
cm_hp_clw = confusion_matrix(y_test, y_predict_hp_clw)
cm_hp_clw
```

Out[121…

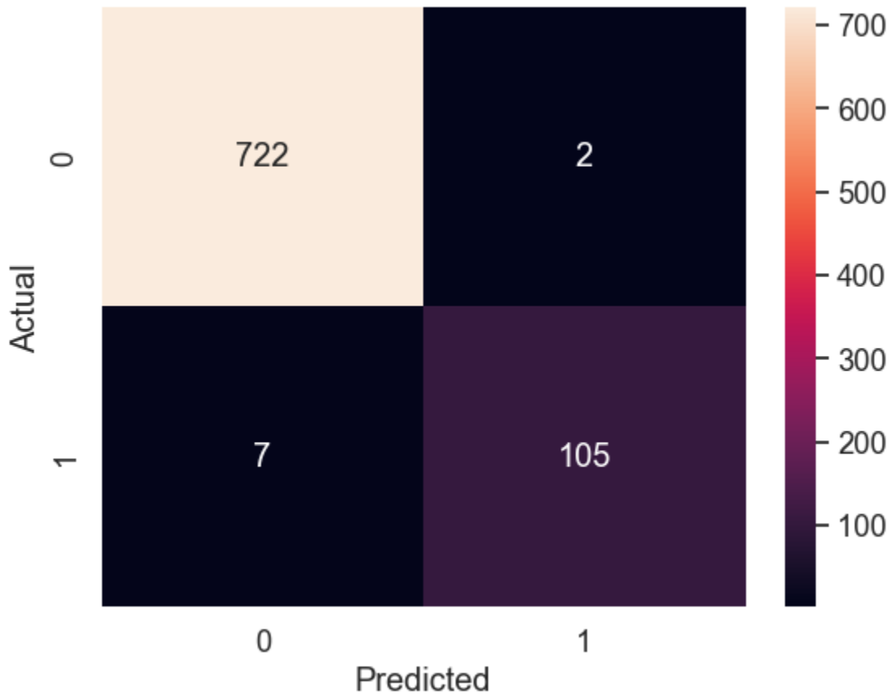
```
array([[722,   2],
       [  7, 105]], dtype=int64)
```

In [122…

```
sns.heatmap(cm_hp_clw, annot=True, fmt = 'd')
plt.xlabel('Predicted')
plt.ylabel('Actual')
```

Out[122…

```
Text(43.25, 0.5, 'Actual')
```



Additional Metrics (ROC-AUC) for BERT

Table of Contents

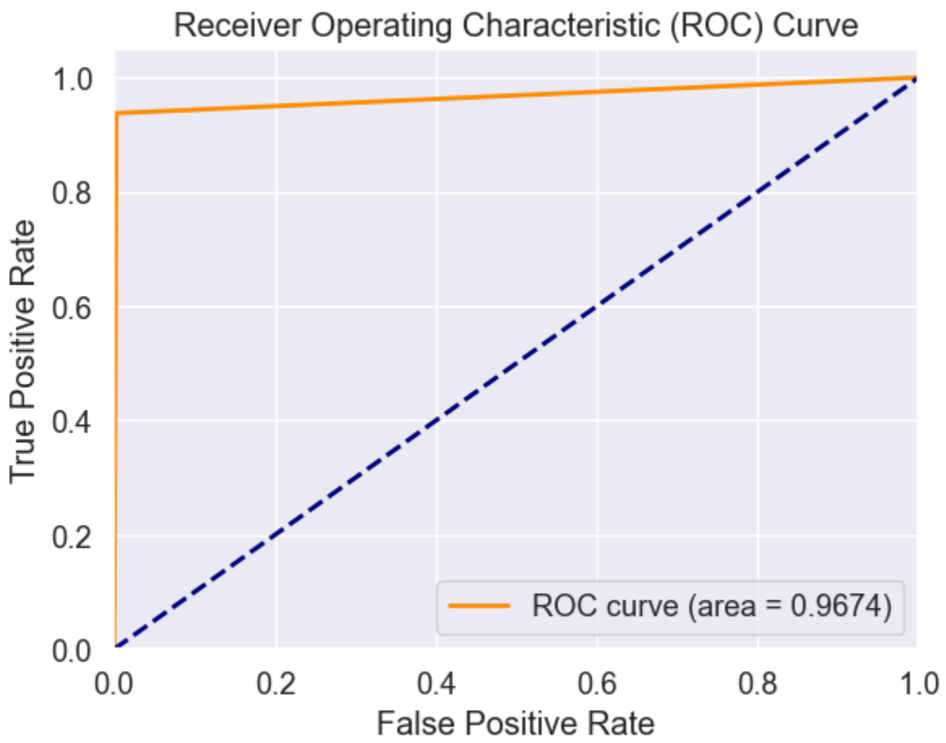
In [123…

```
roc_auc_test = roc_auc_score(y_test, y_predict_hp_clw)
print(f'ROC-AUC Score on Test Set: {roc_auc_test:.4f}')
```

```
ROC-AUC Score on Test Set: 0.9674
```

In [124…

```
# Assuming y_test and y_predict_hp are already defined
fpr, tpr, _ = roc_curve(y_test, y_predict_hp_clw)
roc_auc = auc(fpr, tpr)

# Plotting the ROC curve
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()
```



In [125…

```
# Detection of examples
clw_inf_hp = best_model_clw.predict(reviews)
clw_inf_hp
```

```
1/1 [==============================] - 1s 772ms/step
```

Out[125…

```
array([[0.04036551],
       [0.44526905],
       [0.5218786 ],
       [0.5759099 ],
       [0.00858849]], dtype=float32)
```

Custom Loss

Table of Contents

In [126…

```
%%time
cus_model = tf.keras.models.clone_model(model)  # Cloning the custom model architectire without learned weights
```

```
CPU times: total: 10.2 s
Wall time: 10.7 s
```

In [127…

```
def custom_loss(y_train, y_predict):
    # Convert y_train to float
    y_train = tf.cast(y_train, tf.float32)

    # Apply label smoothing
    y_train_smoothed = y_train * 0.9 + 0.05

    # Compute binary cross-entropy
    bce_loss = tf.keras.losses.binary_crossentropy(y_train_smoothed, y_predict, from_logits=False)
```

```
    return bce_loss
```

In [128…

```
%%time
cus_model.compile(optimizer=Adam(learning_rate=0.001),
                  loss = custom_loss,
                  metrics = metrics)
```

```
CPU times: total: 0 ns
Wall time: 6 ms
```

Model Building

Table of Contents

In [129…

```
%%time
# Fit the model
history_cus = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_val, y_val),
                    callbacks=[early_stopping, lr_scheduler])
```

```
Epoch 1/20
122/122 [==============================] - 63s 514ms/step - loss: 0.0723 - accuracy: 0.9741 - precision: 0.9089 - recall: 0.8967 - val_loss: 0.0396 - val_accuracy: 0.9880 - val_precision: 0.97
22 - val_recall: 0.9375 - lr: 4.0000e-05
Epoch 2/20
122/122 [==============================] - 62s 512ms/step - loss: 0.0657 - accuracy: 0.9769 - precision: 0.9237 - recall: 0.9025 - val_loss: 0.0398 - val_accuracy: 0.9880 - val_precision: 0.99
04 - val_recall: 0.9196 - lr: 4.0000e-05
Epoch 3/20
122/122 [==============================] - 62s 511ms/step - loss: 0.0630 - accuracy: 0.9790 - precision: 0.9265 - recall: 0.9159 - val_loss: 0.0399 - val_accuracy: 0.9868 - val_precision: 0.98
10 - val_recall: 0.9196 - lr: 4.0000e-05
Epoch 4/20
122/122 [==============================] - 62s 512ms/step - loss: 0.0681 - accuracy: 0.9762 - precision: 0.9249 - recall: 0.8948 - val_loss: 0.0398 - val_accuracy: 0.9880 - val_precision: 0.99
04 - val_recall: 0.9196 - lr: 8.0000e-06
CPU times: total: 3min 2s
Wall time: 4min 10s
```

Model Evaluation

Table of Contents

In [130…

```
# Evaluate the model
results_cus = model.evaluate(X_test, y_test)
print(f"Test Loss: {results_cus[0]}")
print(f"Test Accuracy: {results_cus[1]}")
```

```
27/27 [==============================] - 11s 410ms/step - loss: 0.0333 - accuracy: 0.9904 - precision: 0.9815 - recall: 0.9464
Test Loss: 0.033295609056949615
Test Accuracy: 0.9904305934906006
```

In [131…

```
y_predict_cus = cus_model.predict(X_test)
y_predict_cus = y_predict_cus.flatten()
```

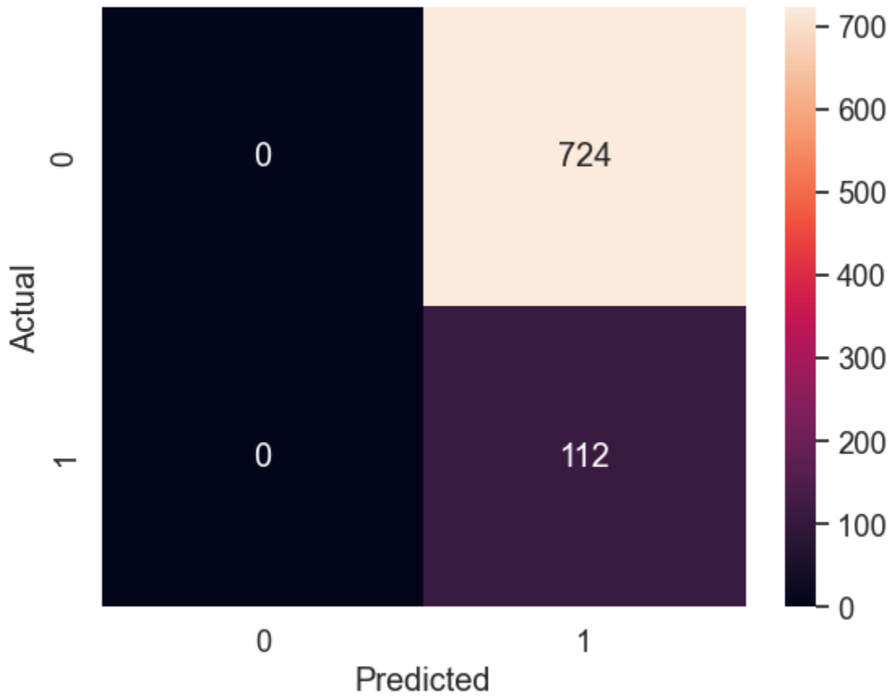
```
27/27 [==============================] - 11s 410ms/step
```

In [132…

```
y_predict_cus = np.where(y_predict_cus > 0.5,1,0)
y_predict_cus
```

Out[132…

```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       
```



In [135…

```
cus = classification_report(y_test, y_predict_cus, target_names=['Ham', 'Spam'])
print(cus)
```

```
              precision    recall  f1-score   support

         Ham       0.00      0.00      0.00       724
        Spam       0.13      1.00      0.24       112

    accuracy                           0.13       836
   macro avg       0.07      0.50      0.12       836
weighted avg       0.02      0.13      0.03       836
```

In [136…

```
# Detection of examples
cus_inf = cus_model.predict(reviews)
cus_inf
```

```
1/1 [==============================] - 1s 754ms/step
```

Out[136…

```
array([[0.64688647],
       [0.65923333],
       [0.5809246 ],
       [0.651914  ],
       [0.57906115]], dtype=float32)
```

Hyperparameter Tuning with Keras Tuner (Custom Loss)

Table of Contents

In [137…

```
%%time
# Initialize Keras Tuner with the existing model
tuner = kt.Hyperband(
```

```
    compile_model,
    objective='val_accuracy',
    max_epochs=5,
    factor=3,
    directory=os.getcwd(),
    project_name='nml_model'
)

# Perform hyperparameter tuning
tuner.search(X_train, y_train, epochs=5, validation_data=(X_val, y_val))

# Get the best model
best_model_cus = tuner.get_best_models(num_models=1)[0]
```

```
Reloading Tuner from C:\Users\sathi\Career\Internship\AFAME\Spam SMS Detection\BERT\nml_model\tuner0.json
CPU times: total: 1.41 s
Wall time: 1.65 s
```

In [138…

```
# Evaluate the best model on the validation set
hp_results_val_cus = best_model_cus.evaluate(X_val, y_val)
print(f"\nValidation Loss: {hp_results_val_cus[0]}")
print(f"Validation Accuracy: {hp_results_val_cus[1]}")
```

```
27/27 [==============================] - 12s 404ms/step - loss: 0.0516 - accuracy: 0.9856 - precision: 0.9505 - recall: 0.9420

Validation Loss: 0.05157732218503952
Validation Accuracy: 0.9856459498405457
```

In [139…

```
# Evaluate the best model on the test set
hp_results_val_cus = best_model_cus.evaluate(X_test, y_test)
print(f"\Test Loss: {hp_results_val_cus[0]}")
print(f"Test Accuracy: {hp_results_val_cus[1]}")
```

```
27/27 [==============================] - 11s 406ms/step - loss: 0.0432 - accuracy: 0.9880 - precision: 0.9554 - recall: 0.9554
\Test Loss: 0.04315700754523277
Test Accuracy: 0.9880383014678955
```

Model Building

Table of Contents

In [140…

```
%%time
# Fitting best model on X_train, y_train, X_val, y_val, X_test, y_test
best_model_cus.fit(X_train, y_train, epochs=20,
                   validation_data=(X_val, y_val),
                   callbacks=[early_stopping, lr_scheduler])
```

```
Epoch 1/20
122/122 [==============================] - 66s 515ms/step - loss: 0.0726 - accuracy: 0.9733 - precision: 0.9068 - recall: 0.8929 - val_loss: 0.0811 - val_accuracy: 0.9749 - val_precision: 0.95
96 - val_recall: 0.8482 - lr: 0.0010
Epoch 2/20
122/122 [==============================] - 63s 515ms/step - loss: 0.0819 - accuracy: 0.9721 - precision: 0.9224 - recall: 0.8642 - val_loss: 0.0546 - val_accuracy: 0.9833 - val_precision: 0.98
04 - val_recall: 0.8929 - lr: 0.0010
Epoch 3/20
122/122 [==============================] - 62s 513ms/step - loss: 0.0739 - accuracy: 0.9741 - precision: 0.9289 - recall: 0.8738 - val_loss: 0.0586 - val_accuracy: 0.9809 - val_precision: 0.89
34 - val_recall: 0.9732 - lr: 0.0010
Epoch 4/20
122/122 [==============================] - 62s 512ms/step - loss: 0.0725 - accuracy: 0.9744 - precision: 0.9123 - recall: 0.8948 - val_loss: 0.0933 - val_accuracy: 0.9713 - val_precision: 1.00
00 - val_recall: 0.7857 - lr: 0.0010
Epoch 5/20
122/122 [==============================] - 63s 515ms/step - loss: 0.0680 - accuracy: 0.9769 - precision: 0.9482 - recall: 0.8757 - val_loss: 0.0433 - val_accuracy: 0.9856 - val_precision: 0.98
08 - val_recall: 0.9107 - lr: 2.0000e-04
Epoch 6/20
122/122 [==============================] - 62s 511ms/step - loss: 0.0624 - accuracy: 0.9782 - precision: 0.9294 - recall: 0.9063 - val_loss: 0.0440 - val_accuracy: 0.9880 - val_precision: 0.95
54 - val_recall: 0.9554 - lr: 2.0000e-04
Epoch 7/20
122/122 [==============================] - 62s 512ms/step - loss: 0.0657 - accuracy: 0.9759 - precision: 0.9198 - recall: 0.8987 - val_loss: 0.0413 - val_accuracy: 0.9844 - val_precision: 0.98
06 - val_recall: 0.9018 - lr: 2.0000e-04
Epoch 8/20
122/122 [==============================] - 63s 514ms/step - loss: 0.0587 - accuracy: 0.9803 - precision: 0.9478 - recall: 0.9025 - val_loss: 0.0410 - val_accuracy: 0.9856 - val_precision: 0.98
08 - val_recall: 0.9107 - lr: 2.0000e-04
Epoch 9/20
122/122 [==============================] - 63s 513ms/step - loss: 0.0606 - accuracy: 0.9769 - precision: 0.9321 - recall: 0.8929 - val_loss: 0.0398 - val_accuracy: 0.9868 - val_precision: 0.98
10 - val_recall: 0.9196 - lr: 2.0000e-04
Epoch 10/20
122/122 [==============================] - 63s 514ms/step - loss: 0.0632 - accuracy: 0.9772 - precision: 0.9323 - recall: 0.8948 - val_loss: 0.0416 - val_accuracy: 0.9844 - val_precision: 0.98
06 - val_recall: 0.9018 - lr: 2.0000e-04
Epoch 11/20
122/122 [==============================] - 63s 521ms/step - loss: 0.0624 - accuracy: 0.9782 - precision: 0.9228 - recall: 0.9140 - val_loss: 0.0437 - val_accuracy: 0.9856 - val_precision: 0.99
02 - val_recall: 0.9018 - lr: 2.0000e-04
Epoch 12/20
122/122 [==============================] - 63s 520ms/step - loss: 0.0591 - accuracy: 0.9787 - precision: 0.9453 - recall: 0.8929 - val_loss: 0.0380 - val_accuracy: 0.9880 - val_precision: 0.98
11 - val_recall: 0.9286 - lr: 4.0000e-05
Epoch 13/20
122/122 [==============================] - 63s 513ms/step - loss: 0.0588 - accuracy: 0.9779 - precision: 0.9468 - recall: 0.8853 - val_loss: 0.0377 - val_accuracy: 0.9868 - val_precision: 0.98
10 - val_recall: 0.9196 - lr: 4.0000e-05
Epoch 14/20
122/122 [==============================] - 62s 511ms/step - loss: 0.0608 - accuracy: 0.9787 - precision: 0.9365 - recall: 0.9025 - val_loss: 0.0383 - val_accuracy: 0.9880 - val_precision: 0.98
11 - val_recall: 0.9286 - lr: 4.0000e-05
Epoch 15/20
122/122 [==============================] - 62s 512ms/step - loss: 0.0631 - accuracy: 0.9797 - precision: 0.9302 - recall: 0.9178 - val_loss: 0.0391 - val_accuracy: 0.9856 - val_precision: 0.98
08 - val_recall: 0.9107 - lr: 4.0000e-05
Epoch 16/20
122/122 [==============================] - 62s 513ms/step - loss: 0.0601 - accuracy: 0.9797 - precision: 0.9405 - recall: 0.9063 - val_loss: 0.0391 - val_accuracy: 0.9868 - val_precision: 0.98
10 - val_recall: 0.9196 - lr: 8.0000e-06
CPU times: total: 12min 12s
Wall time: 16min 44s
```

Out[140…

```
<keras.callbacks.History at 0x21aa51f3e80>
```

Model Evaluation

Table of Contents

In [141…

```
y_predict_hp_cus = best_model_cus.predict(X_test)
y_predict_hp_cus = y_predict_hp_cus.flatten()
```

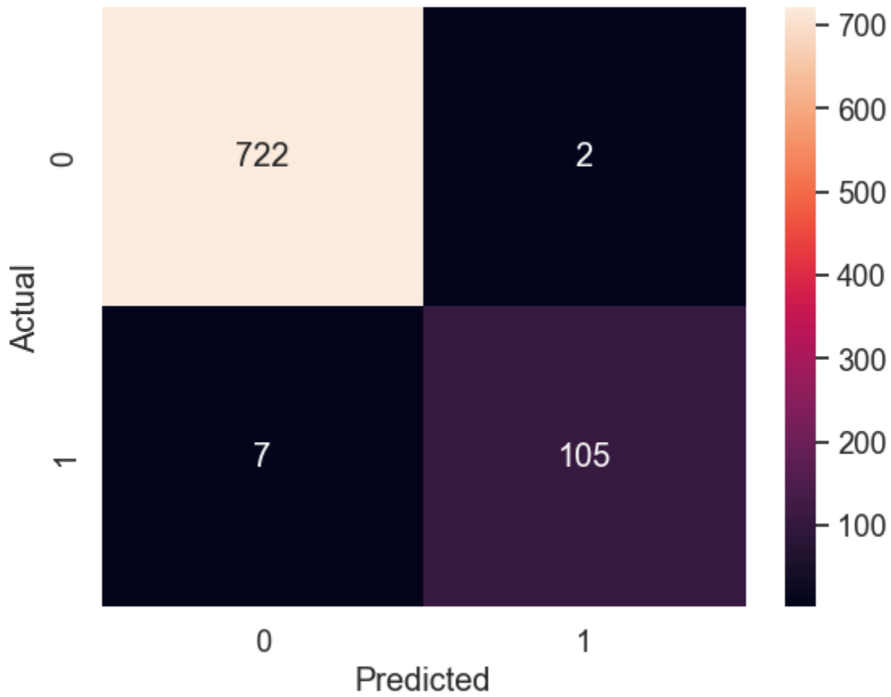
```
27/27 [==============================] - 12s 411ms/step
```

In [142…

```
y_predict_hp_cus = np.where(y_predict_hp_cus > 0.5,1,0)
y_predict_hp_cus
```

Out[142…

```
array([0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0,
       0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0,
       0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
```



In [145…

```
cus_hp = classification_report(y_test, y_predict_hp_cus, target_names=['Ham', 'Spam'])
print(cus_hp)
```

```
              precision    recall  f1-score   support

         Ham       0.99      1.00      0.99       724
        Spam       0.98      0.94      0.96       112

    accuracy                           0.99       836
   macro avg       0.99      0.97      0.98       836
weighted avg       0.99      0.99      0.99       836
```

Additional Metrics (ROC-AUC) for BERT

Table of Contents

In [146…

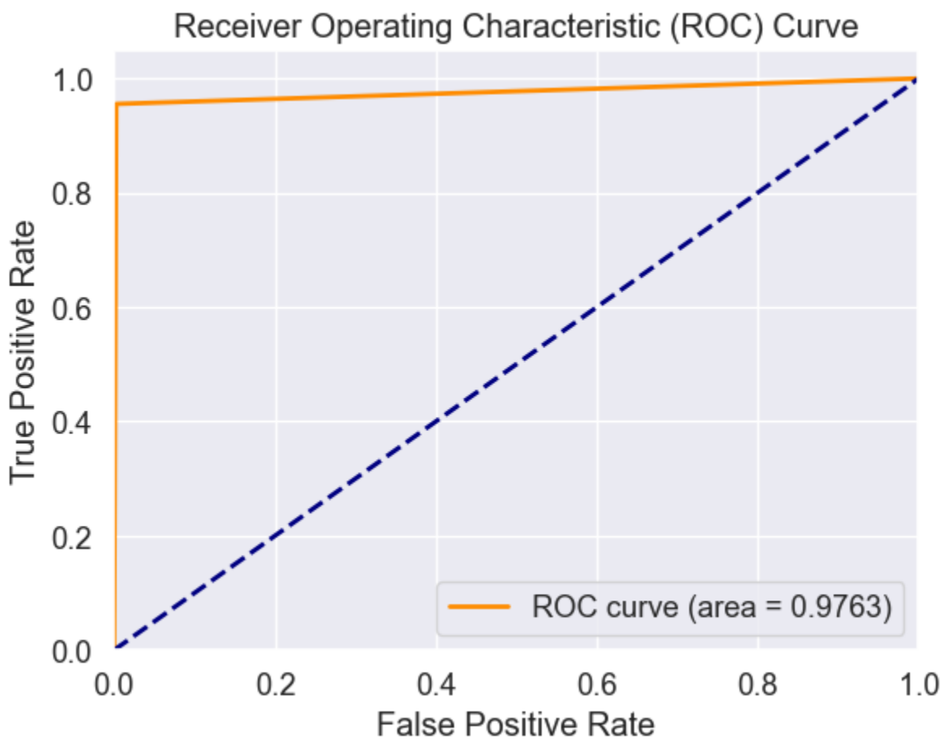
```
roc_auc_test = roc_auc_score(y_test, y_predict_hp_cus)
print(f'ROC-AUC Score on Test Set: {roc_auc_test:.4f}')
```

```
ROC-AUC Score on Test Set: 0.9674
```

In [147…

```
# y_test and y_predict_hp_cus are already defined
fpr, tpr, _ = roc_curve(y_test, y_predict_hp)
roc_auc = auc(fpr, tpr)

# Plotting the ROC curve
plt.figure()
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
```

```
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()
```



In [168…

```
# Detection of examples
cus_inf_hp = best_model_cus.predict(reviews)
cus_inf_hp
```

```
1/1 [==============================] - 0s 69ms/step
```

Out[168…

```
array([[0.0284538 ],
       [0.484665  ],
       [0.2167649 ],
       [0.32992136],
       [0.00383613]], dtype=float32)
```

Performance Comparison of Fine-Tuned BERT

Table of Contents

In [150…

```
# Converting the actual classification report metrics into dictionaries
nml_metrics = classification_report(y_test, y_predict, target_names=['ham', 'spam'], output_dict=True)
dwn_metrics = classification_report(y_test_dwn, y_predict_dwn, target_names=['ham', 'spam'], output_dict=True)
up_metrics = classification_report(y_test_up, y_predict_up, target_names=['ham', 'spam'], output_dict=True)
clw_metrics = classification_report(y_test, y_predict_clw, target_names=['ham', 'spam'], output_dict=True)
cus_metrics = classification_report(y_test, y_predict_cus, target_names=['ham', 'spam'], output_dict=True)

# Create a DataFrame with the relevant metrics
metrics_df = pd.DataFrame({
```

```
    'Model': ['Original', 'Down Sampling', 'Up Sampling', 'Class Weights', 'Custom Loss'],
    'Accuracy': [nml_metrics['accuracy'], dwn_metrics['accuracy'], up_metrics['accuracy'], clw_metrics['accuracy'], cus_metrics['accuracy']],
    'Precision (ham)': [nml_metrics['ham']['precision'], dwn_metrics['ham']['precision'], up_metrics['ham']['precision'], clw_metrics['ham']['precision'], cus_metrics['ham']['precision']],
    'Precision (spam)': [nml_metrics['spam']['precision'], dwn_metrics['spam']['precision'], up_metrics['spam']['precision'], clw_metrics['spam']['precision'], cus_metrics['spam']['precision'
    'Recall (ham)': [nml_metrics['ham']['recall'], dwn_metrics['ham']['recall'], up_metrics['ham']['recall'], clw_metrics['ham']['recall'], cus_metrics['ham']['recall']],
    'Recall (spam)': [nml_metrics['spam']['recall'], dwn_metrics['spam']['recall'], up_metrics['spam']['recall'], clw_metrics['spam']['recall'], cus_metrics['spam']['recall']],
    'F1-score (ham)': [nml_metrics['ham']['f1-score'], dwn_metrics['ham']['f1-score'], up_metrics['ham']['f1-score'], clw_metrics['ham']['f1-score'], cus_metrics['ham']['f1-score']],
    'F1-score (spam)': [nml_metrics['spam']['f1-score'], dwn_metrics['spam']['f1-score'], up_metrics['spam']['f1-score'], clw_metrics['spam']['f1-score'], cus_metrics['spam']['f1-score']],
    'Support (ham)': [nml_metrics['ham']['support'], dwn_metrics['ham']['support'], up_metrics['ham']['support'], clw_metrics['ham']['support'], cus_metrics['ham']['support']],
    'Support (spam)': [nml_metrics['spam']['support'], dwn_metrics['spam']['support'], up_metrics['spam']['support'], clw_metrics['spam']['support'], cus_metrics['spam']['support']]
})

# Display the comprehensive comparison table
metrics_df
```

Out[150…

	Model	Accuracy	Precision (ham)	Precision (spam)	Recall (ham)	Recall (spam)	F1-score (ham)	F1-score (spam)	Support (ham)	Support (spam)
0	Original	0.989234	0.991747	0.972477	0.995856	0.946429	0.993797	0.959276	724.0	112.0
1	Down Sampling	0.977778	0.965517	0.990826	0.991150	0.964286	0.978166	0.977376	113.0	112.0
2	Up Sampling	0.973757	0.969863	0.977716	0.977901	0.969613	0.973865	0.973648	724.0	724.0
3	Class Weights	0.868421	0.876225	0.550000	0.987569	0.098214	0.928571	0.166667	724.0	112.0
4	Custom Loss	0.133971	0.000000	0.133971	0.000000	1.000000	0.000000	0.236287	724.0	112.0

Performance Comparison of Hyperparameter-Tuned BERT

Table of Contents

In [152…

```
# Converting the actual classification report metrics into dictionaries
nml_metrics_hp = classification_report(y_test, y_predict_hp, target_names=['ham', 'spam'], output_dict=True)
dwn_metrics_hp = classification_report(y_test_dwn, y_predict_hp_dwn, target_names=['ham', 'spam'], output_dict=True)
up_metrics_hp = classification_report(y_test_up, y_predict_hp_up, target_names=['ham', 'spam'], output_dict=True)
clw_metrics_hp = classification_report(y_test, y_predict_hp_clw, target_names=['ham', 'spam'], output_dict=True)
cus_metrics_hp = classification_report(y_test, y_predict_hp_cus, target_names=['ham', 'spam'], output_dict=True)

# Create a DataFrame with the relevant metrics
metrics_df = pd.DataFrame({
    'Model': ['Original', 'Down Sampling', 'Up Sampling', 'Class Weights', 'Custom Loss'],
    'Accuracy': [nml_metrics_hp['accuracy'], dwn_metrics_hp['accuracy'], up_metrics_hp['accuracy'], clw_metrics_hp['accuracy'], cus_metrics_hp['accuracy']],
    'Precision (ham)': [nml_metrics_hp['ham']['precision'], dwn_metrics_hp['ham']['precision'], up_metrics_hp['ham']['precision'], clw_metrics_hp['ham']['precision'], cus_metrics_hp['ham']['p
    'Precision (spam)': [nml_metrics_hp['spam']['precision'], dwn_metrics_hp['spam']['precision'], up_metrics_hp['spam']['precision'], clw_metrics_hp['spam']['precision'], cus_metrics_hp['spa
    'Recall (ham)': [nml_metrics_hp['ham']['recall'], dwn_metrics_hp['ham']['recall'], up_metrics_hp['ham']['recall'], clw_metrics_hp['ham']['recall'], cus_metrics_hp['ham']['recall']],
    'Recall (spam)': [nml_metrics_hp['spam']['recall'], dwn_metrics_hp['spam']['recall'], up_metrics_hp['spam']['recall'], clw_metrics_hp['spam']['recall'], cus_metrics_hp['spam']['recall']],
    'F1-score (ham)': [nml_metrics_hp['ham']['f1-score'], dwn_metrics_hp['ham']['f1-score'], up_metrics_hp['ham']['f1-score'], clw_metrics_hp['ham']['f1-score'], cus_metrics_hp['ham']['f1-sco
    'F1-score (spam)': [nml_metrics_hp['spam']['f1-score'], dwn_metrics_hp['spam']['f1-score'], up_metrics_hp['spam']['f1-score'], clw_metrics_hp['spam']['f1-score'], cus_metrics_hp['spam']['
    'Support (ham)': [nml_metrics_hp['ham']['support'], dwn_metrics_hp['ham']['support'], up_metrics_hp['ham']['support'], clw_metrics_hp['ham']['support'], cus_metrics_hp['ham']['support']],
    'Support (spam)': [nml_metrics_hp['spam']['support'], dwn_metrics_hp['spam']['support'], up_metrics_hp['spam']['support'], clw_metrics_hp['spam']['support'], cus_metrics_hp['spam']['suppo
})

# Display the comprehensive comparison table
metrics_df
```

Out[152…

	Model	Accuracy	Precision (ham)	Precision (spam)	Recall (ham)	Recall (spam)	F1-score (ham)	F1-score (spam)	Support (ham)	Support (spam)
0	Original	0.991627	0.993122	0.981651	0.997238	0.955357	0.995176	0.968326	724.0	112.0
1	Down Sampling	0.968889	0.964912	0.972973	0.973451	0.964286	0.969163	0.968610	113.0	112.0
2	Up Sampling	0.977901	0.972678	0.983240	0.983425	0.972376	0.978022	0.977778	724.0	724.0
3	Class Weights	0.989234	0.990398	0.981308	0.997238	0.937500	0.993806	0.958904	724.0	112.0
4	Custom Loss	0.989234	0.990398	0.981308	0.997238	0.937500	0.993806	0.958904	724.0	112.0

Inference Comparison of Fine-Tuned BERT

Table of Contents

In [165…

```
# Reshape arrays to be 1-dimensional
nml_inf = nml_inf.reshape(-1)
dwn_inf = dwn_inf.reshape(-1)
up_inf = up_inf.reshape(-1)
clw_inf = clw_inf.reshape(-1)
cus_inf = cus_inf.reshape(-1)

inferences = {
    'Normal Inference': nml_inf,
    'Down Sampling Inference': dwn_inf,
    'Up Sampling Inference': up_inf,
    'Class Weights Inference': clw_inf,
    'Custom Loss Inference': cus_inf
}

inf = pd.DataFrame(inferences)

# Display the table
inf
```

Out[165…

	Normal Inference	Down Sampling Inference	Up Sampling Inference	Class Weights Inference	Custom Loss Inference
0	0.016572	0.015257	0.083901	0.483630	0.646886
1	0.332832	0.742220	0.743719	0.472468	0.659233
2	0.066134	0.895075	0.828147	0.424398	0.580925
3	0.452054	0.895371	0.873046	0.486227	0.651914
4	0.007615	0.000182	0.000028	0.474624	0.579061

Inference Comparison of Hyperparameter-Tuned BERT

Table of Contents

In [170…

```
# Reshape arrays to be 1-dimensional
nml_inf_hp = nml_inf_hp.reshape(-1)
dwn_inf_hp = dwn_inf_hp.reshape(-1)
up_inf_hp = up_inf_hp.reshape(-1)
```

```
clw_inf_hp = clw_inf_hp.reshape(-1)
cus_inf_hp = cus_inf_hp.reshape(-1)

inferences_hp = {
    'Normal Inference': nml_inf_hp,
    'Down Sampling Inference': dwn_inf_hp,
    'Up Sampling Inference': up_inf_hp,
    'Class Weights Inference': clw_inf_hp,
    'Custom Loss Inference': cus_inf_hp
}

inf_up = pd.DataFrame(inferences_hp)

# Display the table
inf_up
```

Out[170…

	Normal Inference	Down Sampling Inference	Up Sampling Inference	Class Weights Inference	Custom Loss Inference
0	0.016572	0.015257	0.083901	0.483630	0.028454
1	0.332832	0.742220	0.743719	0.472468	0.484665
2	0.066134	0.895075	0.828147	0.424398	0.216765
3	0.452054	0.895371	0.873046	0.486227	0.329921
4	0.007615	0.000182	0.000028	0.474624	0.003836

Conclusion

Table of Contents

Based on the comparison of the fine-tuned BERT model and the hyperparameter-tuned BERT model for spam detection, it is evident that the hyperparameter-tuned BERT model with Up Sampling demonstrates the highest F1-score for spam detection, achieving a value of 0.977778. This indicates a superior balance between precision and recall for identifying spam, making it the most effective model among those tested. In contrast, the fine-tuned BERT model with Down Sampling also performs well, with an F1-score of 0.977376, but it slightly falls short compared to the hyperparameter-tuned model. Other approaches, such as Class Weights and Custom Loss, show significantly lower F1-scores, indicating less effectiveness in spam detection.

Given these results, the hyperparameter-tuned BERT model with Up Sampling is recommended for optimal spam detection performance. This model is well-suited for real industry applications and is ready to be deployed. By saving this best-performing model, robust and reliable spam detection can be ensured in a production environment, enhancing the overall effectiveness and efficiency of the system.

Deployment

Table of Contents

In [171…

```
# Save the entire model to a HDF5 file
best_model_up.save('spam_detection_model.h5')
```

References

Table of Contents

`www.google.com` `www.stackoverflow.com` `www.tensorflowhub.com` `www.geeksforgeeks.com` `www.youtube.com`